

Wham — Frontend Construction Specification

Next.js App Router · SIWE · Arbitrum

- Wham — Frontend Construction Specification
 - 0. Addendum — v1.1 integration corrections
 - 1. How to use this document
 - 2. Stack and architecture decisions
 - 3. Authentication — SIWE
 - 4. Route map
 - 5. Folder structure
 - 6. Shared types
 - 7. Backend API reference (.NET)
 - 8. Smart contract interface reference
 - 9. Page specifications
 - 10. Cross-cutting patterns
 - 11. Build order

Wham — Frontend Construction Specification

Version: 1.1 — includes the \$0 integration addendum aligning this document with the smart contract spec **Target:** Arbitrum Open House Singapore Online Buildathon (Sep 13 - Oct 4, 2026) **Scope:** Complete frontend build plan — every page, its content, the backend endpoints it calls, the contract methods it reads and writes, and mock data for all of it.

0. Addendum — v1.1 integration corrections

The smart contract specification (v1.0) was finalized after this document and resolved several questions this document left open. **Where this addendum conflicts with the body below, the addendum wins.** Everything not mentioned here stands as written.

0.1 `join()` transfers the deposit only — not the first contribution

Body sections affected: §7.4 (`join-intent` response), §8.3 (`join` comment), §9.9 (steps 3–4).

The contract's `join(quote, sig)` pulls **only the collateral**. The first contribution is paid via `contribute(0)` after the circle starts, like every other round. (Reason: a circle that never reaches quorum must refund cleanly with no round accounting to unwind.)

What changes in the join flow (§9.9): - **Step 3 keeps the combined approval** — approve `deposit + first contribution` in one signature, so the user never approves twice. The exactness copy stays truthful: nothing beyond that total can ever be moved. - **Step 4 copy changes.** Replace the “Total due now” framing with two lines: “*Charged now: your 300.00 USDC deposit*” and “*Charged when the circle starts: your first 200.00 USDC contribution*”. The success redirect should point out that the first payment appears on their calendar. - The `requirements` object from `join-intent` keeps its shape; reinterpret `totalDueNow` as the **approval total**, not the transfer amount. Balance checks still validate against the full total — the user needs both amounts.

0.2 New circle parameters: `payoutCollateralBps` and `biddingDuration`

Body sections affected: §8.1 (`CircleParams`), §7.3 (`deployParams`), §9.14 (create flow).

The contract's `CircleParams` struct is now, in exact field order:

```

struct CircleParams {
    address token;
    uint256 contributionPerRound;
    uint8 memberTarget;
    uint32 roundDuration; // seconds
    uint32 biddingDuration; // seconds, < roundDuration; default roundDuration / 4
    uint16 baseCollateralBps;
    uint16 insuranceSkimBps;
    uint16 maxDiscountBps;
    uint16 payoutCollateralBps; // 5000 – see §0.3
    bool biddingEnabled;
    uint64 startsAt;
    address agent;
}

```

`deployParams` from `POST /v1/circles` gains both fields in this order. Neither is user-editable in the create flow: `biddingDuration` defaults to a quarter of the round, `payoutCollateralBps` is fixed at 5000 by the backend. Both are **displayed** in the step-3 review table.

0.3 The payout collateral top-up — new mechanic, three UI changes

When a member wins a round, the contract raises their required collateral to 50% of their remaining obligation and funds the difference **from the payout itself**. A round-0 winner of a 394.00 net pot with 100.00 posted receives 319.00; the 75.00 top-up joins their deposit and is returned at completion. This closes open question #4 from §11.2.

1. **PotBreakdown** gains a row (live and settled): – Deposit top-up (returned at the end) 75.00 .
2. **The bid composer's live panel gains a fourth line**, computed from the new contract read `quoteTopUp(round, address)` : “41.00 USDC would move into your deposit and come back when the circle finishes.” Never show a “you’d receive” figure that ignores the top-up — the preview must equal what settlement pays.
3. **PayoutClaimedMoment** celebrates the amount actually received, with the top-up named beneath it as deferred, not lost.

New contract reads for §8.2: `quoteTopUp(uint8 round, address who) → uint256`, and `leadingBid(round)` is replaced by two calls, `leadingBidder(uint8) → address` and `leadingBidBps(uint8) → uint16`.

0.4 New write: `withdrawRewards()`

Discount redistribution accrues to a per-member rewards balance. Two consequences:

- **Action stack (§9.13) gains a card**: “You’ve earned 24.50 USDC from other members’ discounts” with a withdraw action. Sort it below payout/contribution cards — it is never urgent.
- **contribute() consumes rewards first**. The payment sheet must show the netting: “8.50 USDC of your discount earnings will be used. 41.50 USDC from your wallet.” The approval amount for a contribution is the wallet remainder, not the full C.

The dashboard `PositionStrip` gains the rewards balance as a fourth figure or a chip on the received figure — designer’s choice, but it must be visible without opening a circle.

0.5 Error map additions (§8.3)

Custom error	Shown to user
<code>QuoteTtlTooLong()</code>	“That quote isn’t valid. Get a fresh one.”
<code>QuoteSubjectMismatch()</code> / <code>QuoteCircleMismatch()</code>	“This quote was issued for a different wallet or circle. Get a fresh one.”
<code>FeeOnTransferToken()</code>	“This token isn’t supported.”
<code>PayoutNotOwed()</code>	“There’s no payout waiting for this round.”
<code>QuorumStillPossible()</code>	“This circle can still fill — it can’t be cancelled yet.”

0.6 Open questions from §11.2 — now resolved

#	Resolution
1 · Collateral top-up for delinquents	Cut the “Top up my deposit” CTA . The contract does not expose it in v1; the payout-contingent top-up (§0.3) addresses the underlying risk instead.

2 · Who calls <code>settleRound</code>	Backend keeper primary; keep the member-facing fallback button once <code>contributionDueAt</code> passes.
3 · Bid ties	Impossible on-chain: bids must improve by $\geq 0.25\%$. The slider's 0.25% step is now load-bearing — do not make it finer.
4 · Payout-then-default	Resolved by \$0.3.
5 · Invite-code join	Still open on the backend side: send <code>inviteCode</code> in the <code>join-intent</code> body for private circles.

1. How to use this document

This is a construction document, not a design document. It assumes the homepage visual direction is already settled (trustworthy, warm, modern-banking — Wise/Monzo, not DeFi-neon) and specifies what has to exist behind it.

Each page section follows the same shape:

Field	Meaning
Route	Next.js App Router path
Auth	<code>public</code> , <code>session</code> (SIWE cookie required), or <code>session + membership</code>
Purpose	What the user is trying to do here
Content	Actual sections and copy — write this, don't invent new copy
Reads	Backend endpoints + contract view calls
Writes	Contract transactions + backend mutations
States	Loading / empty / error / edge cases that must be handled

Every type referenced is defined in §6. Every endpoint is defined in §7. Every contract method is defined in §8. If a page references something not in those sections, it is a gap — flag it.

Naming convention used throughout: amounts are always in **base units as decimal strings** (USDC = 6 decimals, so `"2500000000"` = 250 USDC). Never use JavaScript numbers for money. Format at the render layer only.

2. Stack and architecture decisions

2.1 Chosen stack

Layer	Choice	Reason
Framework	Next.js 15, App Router, TypeScript strict	Server components for public/SEO pages, client islands for wallet interaction
Styling	Tailwind CSS v4 + CSS custom properties for the palette	Design tokens live in CSS vars so the banking palette is swappable
Wallet	wagmi v2 + viem v2	Type-safe contract calls, generated ABI types
Connect UI	RainbowKit	Hackathon-speed; skinned to the Wham palette, not left default
Auth	SIWE (EIP-4361) → JWT issued by the .NET backend → httpOnly cookie set by a Next.js route handler	Backend owns identity; frontend never holds a raw token in JS
Server state	TanStack Query v5	Cache invalidation on tx confirmation is the core pattern
Client state	Zustand (small — tx queue, toast queue, filter drafts only)	Everything else is server state
Forms	React Hook Form + Zod	Zod schemas double as API response validators
Charts	Recharts	Reputation history, insurance pool, agent accuracy
Dates	date-fns + date-fns-tz	Circles are cross-timezone by nature; render

		in local, store UTC
Money	dnum (or viem <code>formatUnits / parseUnits</code>)	No float math anywhere
Notifications	Sonner toasts + backend-driven notification centre	

2.2 Rendering strategy per surface

Surface	Rendering
<code>/</code> , <code>/how-it-works</code> , <code>/agent</code>	Static / ISR (60s) — SEO matters, judges will land here
<code>/circles browse</code>	Server component shell + client filter island, ISR 30s
<code>/circles/[id]</code>	Server-rendered detail + client islands for bid/join/contribute
All <code>/app/*</code> authed routes	Client-side with TanStack Query, session-gated in middleware

2.3 The BFF boundary

Next.js route handlers under `/api/bff/*` exist **only** for:

- SIWE nonce/verify/logout (so the JWT lands in an httpOnly cookie)
- Session read (`/api/bff/session`)

All other data goes browser → .NET API directly, with the JWT attached from an httpOnly cookie via a same-origin rewrite. Configure `next.config.ts` to rewrite `/api/v1/*` → `${WHAM_API_URL}/v1/*` so cookies are first-party and CORS never becomes a hackathon-day fire.

```
// next.config.ts
async rewrites() {
  return [{ source: '/api/v1/:path*', destination: `${process.env.WHAM_API_URL}/v1/:path*` }];
}
```

2.4 Chain configuration

Env	Chain	Chain ID	Token
Production	Arbitrum One	42161	USDC 0xaf88d065e77c8cC2239327C5EDb3A432268e5831
Demo / judging	Arbitrum Sepolia	421614	Mock USDC 0x75faf114eafb1BDbe2F0316DF893fd58CE46AA4d

Build and demo on **Arbitrum Sepolia**. Ship a faucet button in the demo build (`/app/faucet`, hidden on mainnet) that mints 5,000 mock USDC — judges will not go hunting for testnet stablecoins.

Contract addresses (placeholders — replace post-deploy):

```
export const CONTRACTS = {
  421614: {
    circleFactory: '0x1F4a2B9c3D8e5A7f0B1c2D3e4F5a6B7c8D9e0F1a',
    reputation: '0x2A5b3C0d4E9f6B8a1C2d3E4f5A6b7C8d9E0f1A2b',
    insurancePool: '0x3B6c4D1e5F0a7C9b2D3e4F5a6B7c8D9e0F1a2B3c',
    agentIdentity: '0x4C7d5E2f6A1b8D0c3E4f5A6b7C8d9E0f1A2b3C4d', // ERC-8004 IdentityRegistry
    agentReputation: '0x5D8e6F3a7B2c9E1d4F5a6B7c8D9e0F1a2B3c4D5e', // ERC-8004 ReputationRegistry
    usdc: '0x75faf114eafb1BDbe2F0316DF893fd58CE46AA4d',
  },
} as const;
```

3. Authentication — SIWE

3.1 Flow

1. User clicks "Connect wallet" → RainbowKit modal, wagmi connect
2. Wallet connected, no session → app calls GET /api/bff/nonce
3. BFF calls GET /v1/auth/nonce (.NET) → { nonce, issuedAt, expiresAt }
4. Frontend builds SIWE message (siwe pkg), calls signMessage
5. POST /api/bff/verify { message, signature }
6. BFF forwards to POST /v1/auth/verify → .NET recovers address, checks nonce + domain + chainId + expiry, returns { token, expiresAt, user }
7. BFF sets httpOnly, Secure, SameSite=Lax cookie `wham\_session`
8. Subsequent /api/v1/* calls carry the cookie via the same-origin rewrite
9. New wallet (firstLogin: true) → redirect to /app/onboarding
- Returning wallet → redirect to intended route or /app

3.2 SIWE message fields

```
const message = new SiweMessage({
  domain: window.location.host,
  address: checksummedAddress,
  statement: 'Sign in to Wham. This signature proves you control this wallet. It does not authorise any transaction or move any funds.',
  uri: window.location.origin,
  version: '1',
  chainId: 421614,
  nonce, // from backend
  issuedAt,
  expirationTime, // nonce + 10 minutes
});
```

The `statement` copy is user-facing and load-bearing — the target user is nervous about signing things. Do not shorten it.

3.3 Session rules

- Cookie: `wham_session`, httpOnly, Secure, SameSite=Lax, 7-day expiry, sliding refresh on any authenticated call within 24h of expiry.
- `middleware.ts` guards `/app/*`: no cookie → redirect to `?signin=1&next=<path>`.
- **Account switch in wallet ≠ session switch.** wagmi `onAccountChange` must clear the session (`POST /api/bff/logout`) and re-prompt. A stale session bound to a different address is the single nastiest bug class here — write this handler before you write any page.
- **Chain switch** to an unsupported chain → global non-dismissable banner: “*Wham runs on Arbitrum. Switch network to continue.*” with a `switchChain` button. Reads keep rendering from cache; all writes disabled.
- Disconnect wallet → clear session, return to `/`.

3.4 Auth-adjacent UI states to build

State	Treatment
Not connected	Primary CTA everywhere reads “Connect wallet”
Connected, not signed in	CTA reads “Sign in to continue” + one-line explainer that signing is free
Signed in, wrong chain	Global banner, writes disabled
Signed in, session expired mid-session	Silent re-prompt modal, preserve the in-progress form
Signature rejected	Inline, non-alarming: “Sign-in cancelled. Nothing was sent.”

4. Route map

Public

Route	Page
<code>/</code>	Landing
<code>/how-it-works</code>	Mechanics explainer

Public endpoints	Accounting endpoints
/circles	Circle browser
/circles/[circleId]	Circle detail (public read view)
/agent	Agent transparency hub
/agent/decisions/[decisionId]	Single decision breakdown (shareable)
/members/[address]	Public reputation profile
/insurance	Insurance pool transparency

Authenticated (/app/* , SIWE session required)

Route	Page
/app	Dashboard
/app/onboarding	First-run: name, notifications, starter circle match
/app/circles	My circles (active / forming / completed)
/app/circles/[circleId]	Member view of a circle (bidding, contributing, payout)
/app/circles/[circleId]/join	Join + collateral flow (multi-step)
/app/circles/create	Create a circle
/app/calendar	Payment calendar
/app/reputation	My reputation and standing
/app/activity	Transaction + notification history
/app/settings	Profile, notifications, connected channels
/app/faucet	(testnet only) Mock USDC faucet

Route groups

```
app/
├─ (marketing)/      # public, static-ish, marketing chrome
│   ├─ page.tsx
│   ├─ how-it-works/
│   ├─ agent/
│   ├─ insurance/
│   └─ members/[address]/
├─ (browse)/         # public, app-lite chrome
│   └─ circles/
│       └─ circles/[circleId]/
├─ app/              # authed, full app chrome + sidebar
│   └─ ...
└─ api/bff/          # SIWE only
```

5. Folder structure

```

src/
├─ app/                # routes (above)
├─ components/
│  ├─ ui/              # primitives: Button, Card, Sheet, Dialog, Tabs...
│  ├─ money/           # Amount, AmountInput, TokenBadge
│  ├─ circle/          # CircleCard, RoundTimeline, MemberTable,
│  │                  # PotBreakdown, BidLadder, JoinStepper
│  ├─ agent/           # FactorBreakdown, RiskBandChip, AgentSeal,
│  │                  # DecisionCard, AccuracyChart
│  ├─ reputation/      # ScoreDial, TierLadder, ReputationTimeline
│  ├─ tx/              # TxButton, TxStatusToast, ApprovalGate
│  └─ layout/          # AppShell, MarketingNav, Sidebar, ChainBanner
├─ lib/
│  ├─ wagmi.ts          # config, chains, connectors
│  ├─ contracts/        # abis/, addresses.ts, generated types
│  ├─ api/              # typed fetch client + endpoint fns
│  ├─ siwe/             # message builder, session helpers
│  ├─ format/           # money, dates, addresses, bps
│  └─ schema/           # zod schemas mirroring §6 types
├─ hooks/
│  ├─ useCircle.ts      useCircleMembers.ts useRound.ts
│  ├─ useAgentQuote.ts useReputation.ts useObligations.ts
│  ├─ useApproveAndWrite.ts # the two-step ERC-20 pattern
│  └─ useTxLifecycle.ts
├─ types/              # §6, hand-written, no codegen
└─ mocks/              # §7 + §8 fixtures, MSW handlers

```

Build the `mocks/` layer first. Every fixture in §7 and §8 of this document should land in `src/mocks/` as typed constants behind MSW handlers, so the entire frontend can be built and demoed before the .NET service or the contracts are finished. Flip a single env var (`NEXT_PUBLIC_USE MOCKS`) to switch.

6. Shared types

All types live in `src/types/`. Mirror each with a Zod schema in `src/lib/schema/` and validate API responses at the boundary — the .NET backend and the frontend will drift during a three-week build, and you want that to fail loudly in dev, not silently in the demo.

6.1 Primitives

```

export type Address = `0x${string}`;
export type Hex = `0x${string}`;
/** ISO 8601 UTC, e.g. "2026-09-20T14:00:00Z" */
export type IsoDateTime = string;
/** uint256 as a decimal string in token base units. USDC = 6dp. NEVER a number. */
export type BaseUnits = string;
/** Basis points. 10000 = 100%. */
export type Bps = number;

export interface TokenInfo {
  address: Address;
  symbol: string; // "USDC"
  decimals: number; // 6
  chainId: number; // 421614
}

```

6.2 Enums / unions

```

export type CircleStatus =
  | 'forming'      // open, taking members, not started
  | 'active'       // rounds running
  | 'completed'    // all rounds settled, collateral released
  | 'unwinding'    // default event, settling out
  | 'cancelled';   // never reached quorum, refunds issued

export type MemberStatus =
  | 'pending_collateral' // joined off-chain, collateral not yet posted
  | 'active'             // collateral posted, current on payments
  | 'paid_out'           // has received their pot, still owes remaining rounds
  | 'delinquent'         // missed a payment, collateral being drawn
  | 'defaulted'          // collateral exhausted
  | 'completed'          // finished cleanly, collateral withdrawn
  | 'exited';            // left before circle started

export type RoundStatus =
  | 'upcoming'
  | 'bidding'      // bid window open
  | 'funding'      // bid closed, contributions due
  | 'settled'      // pot paid out
  | 'covered';     // settled with an insurance draw

export type RiskBand = 'A' | 'B' | 'C' | 'D';

export type CircleTier = 'starter' | 'standard' | 'trusted';

export type ObligationType =
  | 'contribution'
  | 'bid_window'
  | 'payout'
  | 'collateral_release';

```

6.3 Circle

```

export interface Circle {
  id: string; // uuid, backend identity
  onchainAddress: Address | null; // null while 'forming' pre-deploy
  chainId: number;
  name: string;
  description: string;
  status: CircleStatus;
  visibility: 'public' | 'invite';
  inviteCode: string | null;

  token: TokenInfo;
  contributionPerRound: BaseUnits; // what each member pays each round
  totalRounds: number; // == memberTarget
  memberTarget: number;
  memberCount: number;
  grossPotPerRound: BaseUnits; // contributionPerRound * memberTarget
  roundDurationDays: number;

  tier: CircleTier;
  minReputationScore: number; // 0 for starter circles
  baseCollateralBps: Bps; // 20000 = 2x contribution, before agent adjustment
  insuranceSkimBps: Bps; // 150 = 1.5%
  biddingEnabled: boolean;
  maxDiscountBps: Bps; // caps how aggressive bids can be, e.g. 1500

  currentRoundIndex: number | null; // 0-based; null while forming
  startsAt: IsoDateTime | null;
  endsAt: IsoDateTime | null;
  createdAt: IsoDateTime;
  createdBy: Address;

  stats: {
    roundsSettled: number;
    onTimePaymentRate: number; // 0-1
    totalDistributed: BaseUnits;
    insuranceDrawn: BaseUnits;
    averageDiscountBps: Bps;
  };
}

/** Slim shape returned by list endpoints. */
export type CircleSummary = Pick<Circle,
  | 'id' | 'onchainAddress' | 'name' | 'status' | 'token'
  | 'contributionPerRound' | 'grossPotPerRound' | 'memberTarget'
  | 'memberCount' | 'totalRounds' | 'roundDurationDays' | 'tier'
  | 'minReputationScore' | 'baseCollateralBps' | 'startsAt'
> & {
  /** Present only when the request is authenticated. */
  viewer?: ViewerCircleContext;
};

```

6.4 Viewer context

The single most important type in the app. Every circle-scoped view branches on it, and it is what stops you writing `if (member && member.status === ...)` chains in twelve places.

```

export interface ViewerCircleContext {
  isMember: boolean;
  memberStatus: MemberStatus | null;
  canJoin: boolean;
  cannotJoinReason:
    | null
    | 'circle_full'
    | 'circle_started'
    | 'reputation_too_low'
    | 'already_member'
    | 'insufficient_balance'
    | 'open_delinquency_elsewhere';
  canBid: boolean;
  canContribute: boolean;
  canClaimPayout: boolean;
  canWithdrawCollateral: boolean;
  /** Agent-set requirement for this viewer in this circle, if quoted. */
  quotedCollateral: BaseUnits | null;
  outstandingThisRound: BaseUnits | null;
  nextActionLabel: string | null; // e.g. "Pay round 3" – drives the primary CTA
}

```

6.5 Member

```

export interface CircleMember {
  address: Address;
  ensName: string | null;
  displayName: string | null;
  avatarSeed: string; // deterministic identicon seed
  status: MemberStatus;
  joinedAt: IsoDateTime;

  riskBand: RiskBand;
  reputationScore: number; // 0-1000
  collateralRequired: BaseUnits;
  collateralPosted: BaseUnits;
  collateralSlashed: BaseUnits;
  collateralReleasedAt: IsoDateTime | null;

  roundsPaid: number;
  roundsMissed: number;
  payoutRoundIndex: number | null;
  payoutAmountNet: BaseUnits | null;
  winningDiscountBps: Bps | null;
  agentDecisionId: string | null;
}

```

6.6 Round and bids

```

export interface Round {
  circleId: string;
  index: number;           // 0-based
  status: RoundStatus;

  biddingOpensAt: IsoDateTime;
  biddingClosesAt: IsoDateTime;
  contributionDueAt: IsoDateTime;
  settledAt: IsoDateTime | null;

  grossPot: BaseUnits;
  insuranceSkim: BaseUnits;
  winningBidder: Address | null;
  winningDiscountBps: Bps | null;
  discountAmount: BaseUnits | null; // forgone by the winner
  netPayout: BaseUnits | null;      // grossPot - skim - discountAmount
  redistributionPerMember: BaseUnits | null; // discountAmount split among those still waiting

  contributionsReceived: number;
  contributionsExpected: number;
  shortfall: BaseUnits;
  collateralDrawn: BaseUnits;
  insuranceDrawn: BaseUnits;
  settlementTxHash: Hex | null;
}

export interface Bid {
  id: string;
  circleId: string;
  roundIndex: number;
  bidder: Address;
  displayName: string | null;
  discountBps: Bps;
  impliedNetPayout: BaseUnits;
  placedAt: IsoDateTime;
  txHash: Hex;
  status: 'leading' | 'outbid' | 'won' | 'withdrawn';
}

```

6.7 Agent

```

export interface AgentIdentity {
  agentId: string;           // ERC-8004 agent id
  agentAddress: Address;
  domain: string;           // "agent.wham.finance"
  registryAddress: Address;
  registeredAt: IsoDateTime;
  modelVersion: string;      // "heuristic-v1.2.0"
  modelKind: 'rules' | 'learned'; // 'rules' for the hackathon build – say so plainly
  decisionsIssued: number;
  decisionsResolved: number;
  stats: AgentAccuracy;
}

export interface AgentAccuracy {
  /** 0f members the agent placed in each band, what share defaulted. */
  bandOutcomes: Array<{
    band: RiskBand;
    decisions: number;
    cleanCompletions: number;
    missedPayments: number;

```

```

        defaults: number;
        realisedDefaultRate: number; // 0-1
        predictedDefaultRate: number; // 0-1
    }>;
    /** Share of realised losses that posted collateral fully covered. */
    collateralCoverageRate: number;
    calibrationError: number; // mean abs(predicted - realised)
    lastUpdatedAt: IsoDateTime;
}

export interface AgentFactor {
    key: string; // 'completed_circles'
    label: string; // "Completed circles"
    detail: string; // human sentence, shown verbatim in the UI
    direction: 'increase' | 'decrease' | 'neutral';
    deltaBps: Bps; // signed, applied to the multiplier
    evidence: Array<{ label: string; value: string; href: string | null }>;
}

export interface AgentDecision {
    id: string;
    subject: Address;
    circleId: string;
    circleAddress: Address | null;
    createdAt: IsoDateTime;
    expiresAt: IsoDateTime;

    riskBand: RiskBand;
    baselineMultiplierBps: Bps; // circle's baseCollateralBps
    finalMultiplierBps: Bps; // after factors
    collateralRequired: BaseUnits;
    factors: AgentFactor[];

    agentId: string;
    modelVersion: string;
    attestationTxHash: Hex | null;
    attestationUri: string | null; // IPFS or API URI of the full decision record

    outcome: 'pending' | 'completed_clean' | 'missed_payments' | 'defaulted' | 'expired';
    outcomeResolvedAt: IsoDateTime | null;
}

/** EIP-712 payload the contract verifies in join(). */
export interface SignedAgentQuote {
    decisionId: string;
    agentAddress: Address;
    signature: Hex;
    domain: {
        name: 'WhamAgent';
        version: '1';
        chainId: number;
        verifyingContract: Address; // the circle address
    };
    message: {
        subject: Address;
        circle: Address;
        collateralRequired: BaseUnits;
        riskBand: number; // 0=A, 1=B, 2=C, 3=D
        nonce: string;
        expiresAt: number; // unix seconds
    };
}

```

6.8 Reputation

```
export interface ReputationProfile {
  address: Address;
  ensName: string | null;
  displayName: string | null;
  score: number; // 0-1000
  band: RiskBand;
  tier: CircleTier;
  collateralMultiplierBps: Bps; // what the agent currently charges this wallet

  circlesCompleted: number;
  circlesActive: number;
  roundsPaidOnTime: number;
  roundsPaidLate: number;
  roundsMissed: number;
  defaults: number;
  totalContributed: BaseUnits;
  totalReceived: BaseUnits;
  firstSeenAt: IsoDateTime;

  attestationTokenId: string | null;
  attestationTxHash: Hex | null;

  nextTier: {
    tier: CircleTier;
    circlesNeeded: number;
    cleanRoundsNeeded: number;
    unlocks: string[]; // human strings, rendered as a list
  } | null;

  history: ReputationEvent[];
}

export interface ReputationEvent {
  id: string;
  at: IsoDateTime;
  kind: 'circle_completed' | 'round_paid_on_time' | 'round_paid_late'
    | 'round_missed' | 'default' | 'tier_unlocked' | 'attestation_minted';
  scoreDelta: number;
  scoreAfter: number;
  circleId: string | null;
  circleName: string | null;
  txHash: Hex | null;
}
```

6.9 Obligations, insurance, notifications

```
export interface Obligation {
  id: string;
  circleId: string;
  circleName: string;
  circleAddress: Address | null;
  roundIndex: number;
  type: ObligationType;
  amount: BaseUnits | null;
  dueAt: IsoDateTime;
  status: 'upcoming' | 'due_soon' | 'overdue' | 'paid' | 'auto_deducted' | 'received';
  /** True when missing this will trigger a collateral draw – drives the warning styling. */
  autoDeductRisk: boolean;
  autoDeductAmount: BaseUnits | null;
}
```

```

}

export interface InsurancePool {
  chainId: number;
  address: Address;
  token: TokenInfo;
  totalReserves: BaseUnits;
  committedCoverage: BaseUnits;
  availableCoverage: BaseUnits;
  coverageRatio: number; // available / committed
  skimBps: Bps;
  contributingCircles: number;
  claimsPaidCount: number;
  claimsPaidTotal: BaseUnits;
  recentClaims: Array<{
    id: string;
    circleId: string;
    circleName: string;
    roundIndex: number;
    shortfall: BaseUnits;
    collateralDrawn: BaseUnits;
    insurancePaid: BaseUnits;
    at: IsoDateTime;
    txHash: Hex;
  }>;
  history: Array<{ date: string; reserves: BaseUnits; claims: BaseUnits }>;
}

export interface Notification {
  id: string;
  kind: 'payment_due' | 'payment_overdue' | 'bid_window_open' | 'bid_outbid'
    | 'round_settled' | 'payout_ready' | 'collateral_drawn'
    | 'collateral_released' | 'circle_filled' | 'circle_started'
    | 'reputation_updated';
  title: string;
  body: string;
  circleId: string | null;
  href: string | null;
  createdAt: IsoDateTime;
  readAt: IsoDateTime | null;
  severity: 'info' | 'success' | 'warning' | 'critical';
}

export interface UserProfile {
  address: Address;
  displayName: string | null;
  ensName: string | null;
  avatarSeed: string;
  email: string | null;
  emailVerified: boolean;
  telegramHandle: string | null;
  timezone: string; // IANA, e.g. "Asia/Tehran"
  locale: 'en' | 'fa';
  firstLogin: boolean;
  createdAt: IsoDateTime;
  preferences: {
    notifyPaymentDueHours: number[]; // [72, 24, 4]
    notifyBidWindow: boolean;
    notifyRoundSettled: boolean;
    channelEmail: boolean;
    channelTelegram: boolean;
    channelInApp: boolean;
  };
}

```

```
};  
}
```

6.10 Envelope and errors

Every list endpoint returns:

```
export interface Paginated<T> {  
  items: T[];  
  page: number;  
  pageSize: number;  
  total: number;  
  hasMore: boolean;  
}
```

Every error returns HTTP 4xx/5xx with:

```
export interface ApiError {  
  error: {  
    code: ApiErrorCode;  
    message: string; // safe to show the user verbatim  
    details?: Record<string, unknown>;  
    traceId: string;  
  };  
}  
  
export type ApiErrorCode =  
  | 'UNAUTHENTICATED' | 'SESSION_EXPIRED' | 'ADDRESS_MISMATCH'  
  | 'NONCE_INVALID' | 'SIGNATURE_INVALID'  
  | 'CIRCLE_NOT_FOUND' | 'CIRCLE_FULL' | 'CIRCLE_ALREADY_STARTED'  
  | 'NOT_A_MEMBER' | 'ALREADY_A_MEMBER'  
  | 'REPUTATION_TOO_LOW' | 'OPEN_DELIQUENCY'  
  | 'QUOTE_EXPIRED' | 'QUOTE_NOT_FOUND' | 'AGENT_UNAVAILABLE'  
  | 'BID_WINDOW_CLOSED' | 'BID_ABOVE_MAX' | 'BID_NOT_IMPROVING'  
  | 'ALREADY_PAID_OUT'  
  | 'VALIDATION_FAILED' | 'RATE_LIMITED' | 'INTERNAL';
```

Frontend contract: `message` is always user-safe. Render it. Use `code` to decide whether to offer a recovery action (e.g. `QUOTE_EXPIRED` → “Get a fresh quote” button that re-runs `POST /v1/agent/quote`).

7. Backend API reference (.NET)

Base URL: `/api/v1` (rewritten to the .NET service — see §2.3). Auth: `wham_session` httpOnly cookie → `Authorization: Bearer` injected server-side. All timestamps UTC ISO 8601. All amounts base-unit decimal strings.

7.1 Auth

GET /v1/auth/nonce

Public. Returns a single-use nonce bound to the caller’s session cookie.

```
{  
  "nonce": "8jK2mQ9pXvR4tN7wZ3bL",  
  "issuedAt": "2026-09-20T09:14:02Z",  
  "expiresAt": "2026-09-20T09:24:02Z"  
}
```

POST /v1/auth/verify

Public. Body: `{ message: string, signature: Hex }` — the raw SIWE message and signature.

```

{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIweDdBM2Y...",
  "expiresAt": "2026-09-27T09:14:38Z",
  "user": {
    "address": "0x7A3f9C2e1B4d8A6f0C5e3D9b2A1f4E8c7D6b5A40",
    "displayName": null,
    "ensName": "sepehr.eth",
    "avatarSeed": "7A3f9C2e",
    "email": null,
    "emailVerified": false,
    "telegramHandle": null,
    "timezone": "Asia/Tehran",
    "locale": "en",
    "firstLogin": true,
    "createdAt": "2026-09-20T09:14:38Z",
    "preferences": {
      "notifyPaymentDueHours": [72, 24, 4],
      "notifyBidWindow": true,
      "notifyRoundSettled": true,
      "channelEmail": false,
      "channelTelegram": false,
      "channelInApp": true
    }
  }
}

```

Errors: NONCE_INVALID , SIGNATURE_INVALID , ADDRESS_MISMATCH .

POST /v1/auth/logout → 204

GET /v1/auth/session → UserProfile OR 401 UNAUTHENTICATED

7.2 Circles — discovery

GET /v1/circles

Public (viewer context added when authenticated).

Query params:

Param	Type	Notes
status	forming\ active\ completed	repeatable, default forming
tier	starter\ standard\ trusted	repeatable
minContribution / maxContribution	BaseUnits	
memberTarget	int	repeatable
roundDurationDays	int	repeatable
eligibleOnly	bool	filter to circles the viewer’s reputation clears
sort	starting_soon\ newest\ pot_desc\ pot_asc\ filling_fast	default starting_soon
page , pageSize	int	default 1, 12

Response Paginated<CircleSummary> :

```

{
  "items": [
    {
      "id": "c_9f2a41d0",
      "onchainAddress": "0xA1b2C3d4E5f60718293A4b5C6d7E8f90A1b2C3d4",
      "name": "Steady Six",
      "status": "forming",

```

```

"token": { "address": "0x75faf114eafb18Dbe2F0316DF893fd58CE46AA4d", "symbol": "USDC", "decimals": 6, "chainId": 421614 },
"contributionPerRound": "100000000",
"grossPotPerRound": "600000000",
"memberTarget": 6,
"memberCount": 4,
"totalRounds": 6,
"roundDurationDays": 30,
"tier": "starter",
"minReputationScore": 0,
"baseCollateralBps": 20000,
"startsAt": "2026-10-01T00:00:00Z",
"viewer": {
  "isMember": false,
  "memberStatus": null,
  "canJoin": true,
  "cannotJoinReason": null,
  "canBid": false,
  "canContribute": false,
  "canClaimPayout": false,
  "canWithdrawCollateral": false,
  "quotedCollateral": null,
  "outstandingThisRound": null,
  "nextActionLabel": "Join circle"
}
},
{
  "id": "c_44b7e9c1",
  "onchainAddress": "0xB2c3D4e5F60718293A4b5C6d7E8f90A1b2C3d4E5",
  "name": "Freelancers Monthly",
  "status": "forming",
  "token": { "address": "0x75faf114eafb18Dbe2F0316DF893fd58CE46AA4d", "symbol": "USDC", "decimals": 6, "chainId": 421614 },
  "contributionPerRound": "250000000",
  "grossPotPerRound": "2500000000",
  "memberTarget": 10,
  "memberCount": 9,
  "totalRounds": 10,
  "roundDurationDays": 30,
  "tier": "standard",
  "minReputationScore": 320,
  "baseCollateralBps": 15000,
  "startsAt": "2026-09-28T00:00:00Z",
  "viewer": {
    "isMember": false,
    "memberStatus": null,
    "canJoin": false,
    "cannotJoinReason": "reputation_too_low",
    "canBid": false, "canContribute": false, "canClaimPayout": false,
    "canWithdrawCollateral": false, "quotedCollateral": null,
    "outstandingThisRound": null,
    "nextActionLabel": null
  }
}
},
{
  "id": "c_2e81aa93",
  "onchainAddress": "0xC3d4E5f60718293A4b5C6d7E8f90A1b2C3d4E5f6",
  "name": "Diaspora Circle #3",
  "status": "active",
  "token": { "address": "0x75faf114eafb18Dbe2F0316DF893fd58CE46AA4d", "symbol": "USDC", "decimals": 6, "chainId": 421614 },
  "contributionPerRound": "50000000",
  "grossPotPerRound": "400000000",
  "memberTarget": 8,
  "memberCount": 8,

```

```

    "totalRounds": 8,
    "roundDurationDays": 14,
    "tier": "starter",
    "minReputationScore": 0,
    "baseCollateralBps": 20000,
    "startsAt": "2026-08-15T00:00:00Z",
    "viewer": {
      "isMember": false, "memberStatus": null, "canJoin": false,
      "cannotJoinReason": "circle_started", "canBid": false,
      "canContribute": false, "canClaimPayout": false,
      "canWithdrawCollateral": false, "quotedCollateral": null,
      "outstandingThisRound": null, "nextActionLabel": null
    }
  },
  "page": 1, "pageSize": 12, "total": 27, "hasMore": true
}

```

Circle ids always match `/^c_[0-9a-f]{8}$/`. Validate at the schema boundary.

`GET /v1/circles/recommended`

Session. Returns up to 4 `CircleSummary` matched to the viewer's tier, balance and timezone. Used on `/app` and `/app/onboarding`.

`GET /v1/circles/{circleId}`

Public + viewer context. Returns full `Circle` plus embedded first-screen data:

```

{
  "id": "c_2e81aa93",
  "onchainAddress": "0xC3d4E5f60718293A4b5C6d7E8f90A1b2C3d4E5f6",
  "chainId": 421614,
  "name": "Diaspora Circle #3",
  "description": "Fortnightly circle for a group of eight. Small contributions, fast rotation, built for people sending money home.",
  "status": "active",
  "visibility": "public",
  "inviteCode": null,
  "token": { "address": "0x75faf114eafb1BDbe2F0316DF893fd58CE46AA4d", "symbol": "USDC", "decimals": 6, "chainId": 421614 },
  "contributionPerRound": "50000000",
  "totalRounds": 8,
  "memberTarget": 8,
  "memberCount": 8,
  "grossPotPerRound": "400000000",
  "roundDurationDays": 14,
  "tier": "starter",
  "minReputationScore": 0,
  "baseCollateralBps": 20000,
  "insuranceSkimBps": 150,
  "biddingEnabled": true,
  "maxDiscountBps": 1500,
  "currentRoundIndex": 3,
  "startsAt": "2026-08-15T00:00:00Z",
  "endsAt": "2026-12-05T00:00:00Z",
  "createdAt": "2026-08-01T11:20:00Z",
  "createdBy": "0x7A3f9C2e1B4d8A6f0C5e3D9b2A1f4E8c7D6b5A40",
  "stats": {
    "roundsSettled": 3,
    "onTimePaymentRate": 0.958,
    "totalDistributed": "1164000000",
    "insuranceDrawn": "0",
    "averageDiscountBps": 620
  },
  "viewer": {
    "isMember": true,
    "memberStatus": "active",
    "canJoin": false,
    "cannotJoinReason": "already_member",
    "canBid": true,
    "canContribute": true,
    "canClaimPayout": false,
    "canWithdrawCollateral": false,
    "quotedCollateral": "100000000",
    "outstandingThisRound": "50000000",
    "nextActionLabel": "Pay round 4"
  }
}

```

GET /v1/circles/{circleId}/members → CircleMember[]

```
[
  {
    "address": "0x7A3f9C2e1B4d8A6f0C5e3D9b2A1f4E8c7D6b5A40",
    "ensName": "sepehr.eth", "displayName": "Sepehr", "avatarSeed": "7A3f9C2e",
    "status": "active", "joinedAt": "2026-08-02T08:11:00Z",
    "riskBand": "B", "reputationScore": 410,
    "collateralRequired": "100000000", "collateralPosted": "100000000",
    "collateralSlashed": "0", "collateralReleasedAt": null,
    "roundsPaid": 3, "roundsMissed": 0,
    "payoutRoundIndex": null, "payoutAmountNet": null,
    "winningDiscountBps": null, "agentDecisionId": "d_5cle8a02"
  },
  {
    "address": "0x9B4c1D7e2F3a5B8c0D1e2F3a4B5c6D7e8F9a0B12",
    "ensName": null, "displayName": "Mina", "avatarSeed": "9B4c1D7e",
    "status": "paid_out", "joinedAt": "2026-08-02T09:40:00Z",
    "riskBand": "A", "reputationScore": 690,
    "collateralRequired": "75000000", "collateralPosted": "75000000",
    "collateralSlashed": "0", "collateralReleasedAt": null,
    "roundsPaid": 3, "roundsMissed": 0,
    "payoutRoundIndex": 0, "payoutAmountNet": "352000000",
    "winningDiscountBps": 1100, "agentDecisionId": "d_5cle8a03"
  },
  {
    "address": "0xC5d6E7f8A9b0C1d2E3f4A5b6C7d8E9f0A1b2C3d4",
    "ensName": null, "displayName": "Reza", "avatarSeed": "C5d6E7f8",
    "status": "delinquent", "joinedAt": "2026-08-03T14:02:00Z",
    "riskBand": "D", "reputationScore": 145,
    "collateralRequired": "150000000", "collateralPosted": "150000000",
    "collateralSlashed": "50000000", "collateralReleasedAt": null,
    "roundsPaid": 2, "roundsMissed": 1,
    "payoutRoundIndex": null, "payoutAmountNet": null,
    "winningDiscountBps": null, "agentDecisionId": "d_5cle8a04"
  }
]
```

GET /v1/circles/{circleId}/rounds → Round[]

```
[
  {
    "circleId": "c_2e81aa93", "index": 0, "status": "settled",
    "biddingOpensAt": "2026-08-15T00:00:00Z", "biddingClosesAt": "2026-08-18T00:00:00Z",
    "contributionDueAt": "2026-08-28T00:00:00Z", "settledAt": "2026-08-28T00:12:41Z",
    "grossPot": "400000000", "insuranceSkim": "6000000",
    "winningBidder": "0x9B4c1D7e2F3a5B8c0D1e2F3a4B5c6D7e8F9a0B12",
    "winningDiscountBps": 1100, "discountAmount": "44000000",
    "netPayout": "350000000", "redistributionPerMember": "6285714",
    "contributionsReceived": 8, "contributionsExpected": 8,
    "shortfall": "0", "collateralDrawn": "0", "insuranceDrawn": "0",
    "settlementTxHash": "0xd41f0a8c72b96e3f5a1c8d2e7b04f93a6c5e1d8b2f7a09c34e6b1d5f8a2c7e90"
  },
  {
    "circleId": "c_2e81aa93", "index": 2, "status": "settled",
    "biddingOpensAt": "2026-09-12T00:00:00Z", "biddingClosesAt": "2026-09-15T00:00:00Z",
    "contributionDueAt": "2026-09-25T00:00:00Z", "settledAt": "2026-09-25T00:09:03Z",
    "grossPot": "400000000", "insuranceSkim": "6000000",
    "winningBidder": "0xC5d6E7f8A9b0C1d2E3f4A5b6C7d8E9f0A1b2C3d4",
    "winningDiscountBps": 400, "discountAmount": "16000000",
    "netPayout": "378000000", "redistributionPerMember": "3200000",
    "contributionsReceived": 7, "contributionsExpected": 8,
    "shortfall": "50000000", "collateralDrawn": "50000000", "insuranceDrawn": "0",
    "settlementTxHash": "0x8b2f7a09c34e6b1d5f8a2c7e90d41f0a8c72b96e3f5a1c8d2e7b04f93a6c5e1d"
  },
  {
    "circleId": "c_2e81aa93", "index": 3, "status": "bidding",
    "biddingOpensAt": "2026-09-26T00:00:00Z", "biddingClosesAt": "2026-09-29T00:00:00Z",
    "contributionDueAt": "2026-10-09T00:00:00Z", "settledAt": null,
    "grossPot": "400000000", "insuranceSkim": "6000000",
    "winningBidder": null, "winningDiscountBps": null, "discountAmount": null,
    "netPayout": null, "redistributionPerMember": null,
    "contributionsReceived": 2, "contributionsExpected": 8,
    "shortfall": "0", "collateralDrawn": "0", "insuranceDrawn": "0",
    "settlementTxHash": null
  }
]
```

GET /v1/circles/{circleId}/rounds/{roundIndex}/bids → Bid[]

```
[
  {
    "id": "b_71c0", "circleId": "c_2e81aa93", "roundIndex": 3,
    "bidder": "0x7A3f9C2e1B4d8A6f0C5e3D9b2A1f4E8c7D6b5A40",
    "displayName": "Sepehr", "discountBps": 850,
    "impliedNetPayout": "360000000",
    "placedAt": "2026-09-27T18:22:11Z",
    "txHash": "0x3f5a1c8d2e7b04f93a6c5e1d8b2f7a09c34e6b1d5f8a2c7e90d41f0a8c72b96e",
    "status": "leading"
  },
  {
    "id": "b_71bf", "circleId": "c_2e81aa93", "roundIndex": 3,
    "bidder": "0xE1f2A3b4C5d6E7f8A9b0C1d2E3f4A5b6C7d8E9f0",
    "displayName": "Nasrin", "discountBps": 500,
    "impliedNetPayout": "374000000",
    "placedAt": "2026-09-26T11:05:47Z",
    "txHash": "0x6c5e1d8b2f7a09c34e6b1d5f8a2c7e90d41f0a8c72b96e3f5a1c8d2e7b04f93a",
    "status": "outbid"
  }
]
```

GET /v1/circles/{circleId}/activity → Paginated<ActivityItem>

Indexed contract events, human-readable.

```
{
  "items": [
    { "id": "a_881", "kind": "bid_placed", "actor": "0x7A3f9C2e...5A40", "actorName": "Sepehr",
      "summary": "Bid a 8.5% discount for round 4", "amount": null,
      "at": "2026-09-27T18:22:11Z", "txHash": "0x3f5a1c8d..." },
    { "id": "a_880", "kind": "contribution_paid", "actor": "0x9B4c1D7e...0B12", "actorName": "Mina",
      "summary": "Paid round 4", "amount": "50000000",
      "at": "2026-09-26T20:14:02Z", "txHash": "0x2e7b04f9..." },
    { "id": "a_877", "kind": "collateral_drawn", "actor": "0xC5d6E7f8...C3d4", "actorName": "Reza",
      "summary": "Missed round 3 – 50 USDC drawn from collateral", "amount": "50000000",
      "at": "2026-09-25T00:09:03Z", "txHash": "0x8b2f7a09..." }
  ],
  "page": 1, "pageSize": 20, "total": 64, "hasMore": true
}
```

kind ∈ circle_created | member_joined | collateral_posted | bid_placed | bid_withdrawn | contribution_paid | round_settled | payout_claimed | collateral_drawn | insurance_drawn | collateral_released | circle_completed.

7.3 Circles — mutations

POST /v1/circles — create a draft circle

Session. The backend stores metadata and returns the params the frontend passes to `WhamCircleFactory.createCircle`. The contract deploy is a client transaction; the backend links it on the resulting event.

Request:

```
{
  "name": "Ten Month Ten",
  "description": "Monthly circle, 10 people, 200 USDC each.",
  "contributionPerRound": "200000000",
  "memberTarget": 10,
  "roundDurationDays": 30,
  "biddingEnabled": true,
  "maxDiscountBps": 1500,
  "visibility": "invite",
  "startsAt": "2026-10-15T00:00:00Z",
  "tokenAddress": "0x75faf114eafb1BDbe2F0316DF893fd58CE46AA4d"
}
```

Response 201 :

```
{
  "circle": { "id": "c_a7e30b12", "status": "forming", "onchainAddress": null, "inviteCode": "WHAM-4KQ2-8XLP" },
  "deployParams": {
    "token": "0x75faf114eafb1BDbe2F0316DF893fd58CE46AA4d",
    "contributionPerRound": "200000000",
    "memberTarget": 10,
    "roundDuration": 2592000,
    "baseCollateralBps": 15000,
    "insuranceSkimBps": 150,
    "maxDiscountBps": 1500,
    "biddingEnabled": true,
    "startsAt": 1792108800,
    "salt": "0x9c1e4a7b3d5f8092c6e1b4a7d3f509826c1e4b7a3d5f8092c6e1b4a7d3f50982"
  }
}
```

POST /v1/circles/{circleId}/link-deployment

Session. Body { txHash, onchainAddress }. Backend confirms the tx and flips the circle live. Returns updated `Circle`.

POST /v1/circles/{circleId}/join-intent

Session. Reserves a seat for 15 minutes and triggers an agent quote. This is what the join flow calls at step 1.

Response:

```
{
  "reservationId": "r_3f81c2",
  "expiresAt": "2026-09-27T19:00:00Z",
  "decision": { "...": "AgentDecision – see §7.4" },
  "quote": { "...": "SignedAgentQuote – see §7.4" },
  "requirements": {
    "collateralRequired": "300000000",
    "firstContribution": "200000000",
    "totalDueNow": "500000000",
    "walletBalance": "812400000",
    "sufficientBalance": true,
    "currentAllowance": "0",
    "approvalNeeded": "500000000"
  }
}
```

Errors: `CIRCLE_FULL`, `REPUTATION_TOO_LOW`, `OPEN_DELINQUENCY`, `ALREADY_A_MEMBER`, `AGENT_UNAVAILABLE`.

DELETE /v1/circles/{circleId}/join-intent → 204 (release the seat if the user backs out)

POST /v1/circles/{circleId}/confirm-join

Session. Body { txHash: Hex, reservationId: string }. Backend watches the tx, on success marks the member active. Returns { status: 'pending' | 'confirmed' | 'failed', member: CircleMember | null }.

GET /v1/circles/join/{inviteCode} → CircleSummary (public, resolves an invite link)

7.4 Agent

GET /v1/agent/identity → AgentIdentity

```

{
  "agentId": "8004:421614:1042",
  "agentAddress": "0xA6e70B9c4D1f3E58a2B7c0D9e4F1a6B3c8D5e2F7",
  "domain": "agent.wham.finance",
  "registryAddress": "0x4C7d5E2f6A1b8D0c3E4f5A6b7C8d9E0f1A2b3C4d",
  "registeredAt": "2026-09-14T10:02:00Z",
  "modelVersion": "heuristic-v1.2.0",
  "modelKind": "rules",
  "decisionsIssued": 412,
  "decisionsResolved": 268,
  "stats": {
    "bandOutcomes": [
      { "band": "A", "decisions": 96, "cleanCompletions": 94, "missedPayments": 2, "defaults": 0, "realisedDefaultRate": 0.0,
        "predictedDefaultRate": 0.01 },
      { "band": "B", "decisions": 118, "cleanCompletions": 109, "missedPayments": 8, "defaults": 1, "realisedDefaultRate": 0.008,
        "predictedDefaultRate": 0.03 },
      { "band": "C", "decisions": 41, "cleanCompletions": 33, "missedPayments": 6, "defaults": 2, "realisedDefaultRate": 0.049,
        "predictedDefaultRate": 0.07 },
      { "band": "D", "decisions": 13, "cleanCompletions": 8, "missedPayments": 3, "defaults": 2, "realisedDefaultRate": 0.154,
        "predictedDefaultRate": 0.14 }
    ],
    "collateralCoverageRate": 0.94,
    "calibrationError": 0.021,
    "lastUpdatedAt": "2026-09-27T06:00:00Z"
  }
}

```

Copy note for the UI: wherever `modelKind: "rules"` is rendered, show the badge “Rules-based v1” with tooltip “Wham’s underwriter currently scores on explicit, published rules. Every decision is logged on-chain so its accuracy can be measured — and so a learned model can be held to the same record when it replaces this one.” Do not let the interface imply a trained model exists.

POST /v1/agent/quote

Session. Body { `circleId` }. Returns a fresh `AgentDecision` + `SignedAgentQuote`. Idempotent within the quote TTL.

```

{
  "decision": {
    "id": "d_5c1e8a02",
    "subject": "0x7A3f9C2e1B4d8A6f0C5e3D9b2A1f4E8c7D6b5A40",
    "circleId": "c_a7e30b12",
    "circleAddress": "0xD4e5F60718293A4b5C6d7E8f90A1b2C3d4E5f607",
    "createdAt": "2026-09-27T18:45:00Z",
    "expiresAt": "2026-09-27T19:00:00Z",
    "riskBand": "B",
    "baselineMultiplierBps": 15000,
    "finalMultiplierBps": 15000,
    "collateralRequired": "300000000",
    "factors": [
      {
        "key": "completed_circles",
        "label": "Completed circles",
        "detail": "You've finished 2 circles without missing a payment. That reduces your deposit.",
        "direction": "decrease",
        "deltaBps": -3000,
        "evidence": [
          { "label": "Diaspora Circle #1", "value": "8/8 rounds on time", "href": "/circles/c_11aa2f30" },
          { "label": "Steady Six (Mar)", "value": "6/6 rounds on time", "href": "/circles/c_0b93de41" }
        ]
      },
      {
        "key": "wallet_age",

```

```

    "label": "Wallet history",
    "detail": "This wallet has been active for 14 months, which is above the threshold we treat as established.",
    "direction": "decrease",
    "deltaBps": -1000,
    "evidence": [{ "label": "First activity", "value": "Jul 2025", "href": null }]
  },
  {
    "key": "circle_size_exposure",
    "label": "Circle size",
    "detail": "A 10-person circle at 200 USDC exposes members to a larger pot than your history covers, so we hold a little more.",
    "direction": "increase",
    "deltaBps": 2500,
    "evidence": [
      { "label": "Pot per round", "value": "2,000 USDC", "href": null },
      { "label": "Largest circle completed", "value": "600 USDC pot", "href": null }
    ]
  },
  {
    "key": "concurrent_obligations",
    "label": "Active commitments",
    "detail": "You're currently paying into 1 other circle. Two concurrent commitments is within our comfort range.",
    "direction": "increase",
    "deltaBps": 1500,
    "evidence": [{ "label": "Active circles", "value": "1", "href": "/app/circles" }]
  },
  {
    "key": "no_defaults",
    "label": "Default history",
    "detail": "No defaults or written-off balances on record.",
    "direction": "neutral",
    "deltaBps": 0,
    "evidence": []
  }
],
"agentId": "8004:421614:1042",
"modelVersion": "heuristic-v1.2.0",
"attestationTxHash": "0x5f8a2c7e90d41f0a8c72b96e3f5a1c8d2e7b04f93a6c5e1d8b2f7a09c34e6b1d",
"attestationUri": "https://api.wham.finance/v1/agent/decisions/d_5c1e8a02",
"outcome": "pending",
"outcomeResolvedAt": null
},
"quote": {
  "decisionId": "d_5c1e8a02",
  "agentAddress": "0xA6e70B9c4D1f3E58a2B7c0D9e4F1a6B3c8D5e2F7",
  "signature":
"0x1c8d2e7b04f93a6c5e1d8b2f7a09c34e6b1d5f8a2c7e90d41f0a8c72b96e3f5a1c8d2e7b04f93a6c5e1d8b2f7a09c34e6b1d5f8a2c7e90d41f0a8c72b96e3f5a1b",
  "domain": {
    "name": "WhamAgent", "version": "1", "chainId": 421614,
    "verifyingContract": "0xD4e5F60718293A4b5C6d7E8f90A1b2C3d4E5f607"
  },
  "message": {
    "subject": "0x7A3f9C2e1B4d8A6f0C5e3D9b2A1f4E8c7D6b5A40",
    "circle": "0xD4e5F60718293A4b5C6d7E8f90A1b2C3d4E5f607",
    "collateralRequired": "300000000",
    "riskBand": 1,
    "nonce": "17",
    "expiresAt": 1790707200
  }
}
}
}

```

The collateral formula, stated once so the UI can render it verbatim:

```
collateralRequired = contributionPerRound × (finalMultiplierBps / 10000)
finalMultiplierBps = baselineMultiplierBps +  $\sum$  factor.deltaBps
```

For the mock above: baseline 15000 bps; factors sum to $-3000 - 1000 + 2500 + 1500 + 0 = 0$; final 15000 bps; $200 \text{ USDC} \times 1.5 = 300 \text{ USDC}$. The join screen (\$9.9) must show this as a visible sum, not just the result — the whole point of the agent is that its reasoning is inspectable.

GET /v1/agent/decisions?subject={address}&circleId=&page= → Paginated<AgentDecision>

GET /v1/agent/decisions/{decisionId} → AgentDecision (public — these are shareable, judges will open them)

GET /v1/agent/stats → AgentAccuracy

7.5 Me

GET /v1/me → UserProfile

PATCH /v1/me → body: partial { displayName, email, telegramHandle, timezone, locale, preferences } → UserProfile

GET /v1/me/circles?status=

Returns CircleSummary[] always with viewer populated, grouped client-side.

GET /v1/me/summary — the dashboard payload, one call

```
{
  "totalCommitted": "1150000000",
  "collateralLocked": "400000000",
  "receivedToDate": "350000000",
  "nextObligation": {
    "id": "o_9931", "circleId": "c_2e81aa93", "circleName": "Diaspora Circle #3",
    "circleAddress": "0xC3d4E5f60718293A4b5C6d7E8f90A1b2C3d4E5f6",
    "roundIndex": 3, "type": "contribution", "amount": "50000000",
    "dueAt": "2026-10-09T00:00:00Z", "status": "upcoming",
    "autoDeductRisk": true, "autoDeductAmount": "50000000"
  },
  "openBidWindows": [
    { "circleId": "c_2e81aa93", "circleName": "Diaspora Circle #3", "roundIndex": 3,
      "closesAt": "2026-09-29T00:00:00Z", "myBidBps": 850, "leadingBps": 850, "amLeading": true }
  ],
  "reputation": { "score": 410, "band": "B", "tier": "standard", "delta30d": 45 },
  "activeCircles": 2,
  "completedCircles": 2,
  "walletBalance": "812400000",
  "unreadNotifications": 3
}
```

GET /v1/me/obligations?from=&to= → Obligation[]

```
[
  { "id": "o_9931", "circleId": "c_2e81aa93", "circleName": "Diaspora Circle #3",
    "circleAddress": "0xC3d4E5f60718293A4b5C6d7E8f90A1b2C3d4E5f6", "roundIndex": 3,
    "type": "contribution", "amount": "50000000", "dueAt": "2026-10-09T00:00:00Z",
    "status": "upcoming", "autoDeductRisk": true, "autoDeductAmount": "50000000" },
  { "id": "o_9932", "circleId": "c_2e81aa93", "circleName": "Diaspora Circle #3",
    "circleAddress": "0xC3d4E5f60718293A4b5C6d7E8f90A1b2C3d4E5f6", "roundIndex": 3,
    "type": "bid_window", "amount": null, "dueAt": "2026-09-29T00:00:00Z",
    "status": "due_soon", "autoDeductRisk": false, "autoDeductAmount": null },
  { "id": "o_9940", "circleId": "c_a7e30b12", "circleName": "Ten Month Ten",
    "circleAddress": "0xD4e5F60718293A4b5C6d7E8f90A1b2C3d4E5f607", "roundIndex": 0,
    "type": "contribution", "amount": "200000000", "dueAt": "2026-10-15T00:00:00Z",
    "status": "upcoming", "autoDeductRisk": true, "autoDeductAmount": "200000000" },
  { "id": "o_9955", "circleId": "c_0b93de41", "circleName": "Steady Six (Mar)",
    "circleAddress": "0xE5f60718293A4b5C6d7E8f90A1b2C3d4E5f60718", "roundIndex": 5,
    "type": "collateral_release", "amount": "150000000", "dueAt": "2026-10-20T00:00:00Z",
    "status": "upcoming", "autoDeductRisk": false, "autoDeductAmount": null }
]
```

GET /v1/me/reputation → ReputationProfile

```
{
  "address": "0x7A3f9C2e1B4d8A6f0C5e3D9b2A1f4E8c7D6b5A40",
  "ensName": "sepehr.eth", "displayName": "Sepehr",
  "score": 410, "band": "B", "tier": "standard",
  "collateralMultiplierBps": 15000,
  "circlesCompleted": 2, "circlesActive": 2,
  "roundsPaidOnTime": 17, "roundsPaidLate": 1, "roundsMissed": 0,
  "defaults": 0,
  "totalContributed": "1150000000", "totalReceived": "3500000000",
  "firstSeenAt": "2025-07-04T00:00:00Z",
  "attestationTokenId": "1042",
  "attestationTxHash": "0x90d41f0a8c72b96e3f5a1c8d2e7b04f93a6c5e1d8b2f7a09c34e6b1d5f8a2c7e",
  "nextTier": {
    "tier": "trusted",
    "circlesNeeded": 1,
    "cleanRoundsNeeded": 6,
    "unlocks": [
      "Circles up to 20 members",
      "Deposit as low as 0.75x your contribution",
      "Ability to create standard-tier circles"
    ]
  },
},
"history": [
  { "id": "re_31", "at": "2026-09-25T00:09:03Z", "kind": "round_paid_on_time",
    "scoreDelta": 8, "scoreAfter": 410, "circleId": "c_2e81aa93",
    "circleName": "Diaspora Circle #3", "txHash": "0x8b2f7a09..." },
  { "id": "re_27", "at": "2026-07-30T12:00:00Z", "kind": "circle_completed",
    "scoreDelta": 120, "scoreAfter": 365, "circleId": "c_0b93de41",
    "circleName": "Steady Six (Mar)", "txHash": "0x2c7e90d4..." },
  { "id": "re_26", "at": "2026-07-30T12:00:00Z", "kind": "attestation_minted",
    "scoreDelta": 0, "scoreAfter": 245, "circleId": "c_0b93de41",
    "circleName": "Steady Six (Mar)", "txHash": "0x1f0a8c72..." }
]
}
```

GET /v1/me/notifications?unreadOnly= → Paginated<Notification>

POST /v1/me/notifications/read — body { ids: string[] | 'all' } → 204

7.6 Public profiles, insurance, stats

GET /v1/members/{address} → ReputationProfile (public subset — omits email, obligations)

GET /v1/insurance/pool → InsurancePool

```
{
  "chainId": 421614,
  "address": "0x3B6c4D1e5F0a7C9b2D3e4F5a6B7c8D9e0F1a2B3c",
  "token": { "address": "0x75faf114eafb1BDbe2F0316DF893fd58CE46AA4d", "symbol": "USDC", "decimals": 6, "chainId": 421614 },
  "totalReserves": "48620000000",
  "committedCoverage": "31400000000",
  "availableCoverage": "17220000000",
  "coverageRatio": 0.548,
  "skimBps": 150,
  "contributingCircles": 27,
  "claimsPaidCount": 4,
  "claimsPaidTotal": "1180000000",
  "recentClaims": [
    { "id": "cl_04", "circleId": "c_71ee02aa", "circleName": "Weekly Ten",
      "roundIndex": 6, "shortfall": "400000000", "collateralDrawn": "300000000",
      "insurancePaid": "100000000", "at": "2026-09-18T00:11:00Z",
      "txHash": "0x72b96e3f5a1c8d2e7b04f93a6c5e1d8b2f7a09c34e6bd5f8a2c7e90d41f0a8c" }
  ],
  "history": [
    { "date": "2026-07-01", "reserves": "12400000000", "claims": "0" },
    { "date": "2026-08-01", "reserves": "26800000000", "claims": "380000000" },
    { "date": "2026-09-01", "reserves": "41200000000", "claims": "700000000" },
    { "date": "2026-09-27", "reserves": "48620000000", "claims": "1180000000" }
  ]
}
```

GET /v1/stats — homepage numbers (public, cached 60s)

```
{
  "circlesFormed": 27,
  "activeMembers": 214,
  "totalDistributed": "184300000000",
  "roundsSettled": 143,
  "onTimePaymentRate": 0.967,
  "defaultsCoveredByCollateral": 6,
  "defaultsRequiringInsurance": 1,
  "membersWhoLostFunds": 0
}
```

membersWhoLostFunds: 0 is the single most persuasive number on the site. Feature it.

8. Smart contract interface reference

Five contracts the frontend touches. Generate viem types from the ABIs into `src/lib/contracts/generated.ts` via `wagmi generate` — do not hand-type call signatures.

Contract	Role
WhamCircleFactory	Deploys a minimal-proxy circle per group
WhamCircle	Per-circle escrow: collateral, contributions, bidding, payouts, slashing
WhamReputation	Soulbound attestation, score per address
WhamInsurancePool	Shared reserve, funded by the per-round skim
ERC-8004 IdentityRegistry / ReputationRegistry	Agent identity and decision attestations

Plus standard ERC20 (USDC) for `balanceOf` / `allowance` / `approve`.

8.1 WhamCircleFactory

```
struct CircleParams {
    address token;
    uint256 contributionPerRound;
    uint8 memberTarget;
    uint32 roundDuration; // seconds
    uint16 baseCollateralBps;
    uint16 insuranceSkimBps;
    uint16 maxDiscountBps;
    bool biddingEnabled;
    uint64 startsAt; // unix seconds
    address agent; // ERC-8004 agent address authorised to sign quotes
}

function createCircle(CircleParams calldata p, bytes32 salt) external returns (address circle);
function predictAddress(CircleParams calldata p, bytes32 salt) external view returns (address);
function circlesOf(address creator) external view returns (address[] memory);
function isCircle(address a) external view returns (bool);

event CircleCreated(address indexed circle, address indexed creator, bytes32 salt);
```

Frontend usage: `/app/circles/create` step 3 sends `createCircle(deployParams, salt)` using the exact struct returned by `POST /v1/circles`. Read `predictAddress` first to show the user the address before they sign — a small trust cue that costs nothing.

8.2 WhamCircle — reads

```

enum CircleState { Forming, Active, Completed, Unwinding, Cancelled }
enum MemberState { None, PendingCollateral, Active, PaidOut, Delinquent, Defaulted, Completed, Exited }
enum RoundState { Upcoming, Bidding, Funding, Settled, Covered }

struct Config {
    address token; uint256 contributionPerRound; uint8 memberTarget;
    uint32 roundDuration; uint16 baseCollateralBps; uint16 insuranceSkimBps;
    uint16 maxDiscountBps; bool biddingEnabled; uint64 startsAt; address agent;
}

function config() external view returns (Config memory);
function state() external view returns (CircleState);
function currentRound() external view returns (uint8);

struct Member {
    MemberState state;
    uint256 collateralRequired;
    uint256 collateralPosted;
    uint256 collateralSlashed;
    uint8 roundsPaid;
    uint8 roundsMissed;
    int16 payoutRound; // -1 = not yet paid out
    uint16 winningDiscountBps;
    uint64 joinedAt;
}

function memberOf(address who) external view returns (Member memory);
function memberList() external view returns (address[] memory);
function memberCount() external view returns (uint8);

struct RoundInfo {
    RoundState state;
    uint64 biddingClosesAt;
    uint64 contributionDueAt;
    uint256 grossPot;
    address winner;
    uint16 winningDiscountBps;
    uint256 netPayout;
    uint8 contributionsReceived;
    uint256 shortfall;
    uint256 insuranceDrawn;
}

function roundInfo(uint8 index) external view returns (RoundInfo memory);

function hasContributed(uint8 round, address who) external view returns (bool);
function bidOf(uint8 round, address who) external view returns (uint16 discountBps);
function leadingBid(uint8 round) external view returns (address bidder, uint16 discountBps);
function quotePayout(uint8 round, uint16 discountBps) external view returns (uint256 netPayout);
function amountDue(uint8 round, address who) external view returns (uint256);
function withdrawableCollateral(address who) external view returns (uint256);
function agentNonce(address who) external view returns (uint256);

```

Mock read results for circle `0xC3d4E5f60718293A4b5C6d7E8f90A1b2C3d4E5f6` (Diaspora Circle #3, round 3 in bidding):

```

export const MOCK_CIRCLE_READS = {
  config: {
    token: '0x75faf114eafb1BDbe2F0316DF893fd58CE46AA4d',
    contributionPerRound: 50_000000n,
    memberTarget: 8,
    roundDuration: 1_209_600,           // 14 days
    baseCollateralBps: 20000,
    insuranceSkimBps: 150,
    maxDiscountBps: 1500,
    biddingEnabled: true,
    startsAt: 1786_838_400n,
    agent: '0xA6e70B9c4D1f3E58a2B7c0D9e4F1a6B3c8D5e2F7',
  },
  state: 1,                             // Active
  currentRound: 3,
  memberCount: 8,
  memberOf_viewer: {
    state: 2,                             // Active
    collateralRequired: 100_000000n,
    collateralPosted: 100_000000n,
    collateralSlashed: 0n,
    roundsPaid: 3,
    roundsMissed: 0,
    payoutRound: -1,
    winningDiscountBps: 0,
    joinedAt: 1785_628_260n,
  },
  roundInfo_3: {
    state: 1,                             // Bidding
    biddingClosesAt: 1790_640_000n,
    contributionDueAt: 1791_504_000n,
    grossPot: 400_000000n,
    winner: '0x0000000000000000000000000000000000000000000000000000000000000000',
    winningDiscountBps: 0,
    netPayout: 0n,
    contributionsReceived: 2,
    shortfall: 0n,
    insuranceDrawn: 0n,
  },
  leadingBid_3: {
    bidder: '0x7A3f9C2e1B4d8A6f0C5e3D9b2A1f4E8c7D6b5A40',
    discountBps: 850,
  },
  quotePayout_3_850: 360_000000n,       // 400 - 6 (skim) - 34 (discount)
  amountDue_3_viewer: 50_000000n,
  hasContributed_3_viewer: false,
  withdrawableCollateral_viewer: 0n,
  agentNonce_viewer: 17n,
} as const;

```

8.3 WhamCircle — writes

```

struct AgentQuote {
    address subject;
    address circle;
    uint256 collateralRequired;
    uint8 riskBand; // 0=A ... 3=D
    uint256 nonce;
    uint64 expiresAt;
}

/// Pulls collateralRequired + first contribution. Requires prior ERC20 approve.
function join(AgentQuote calldata q, bytes calldata agentSig) external;

/// Leave before the circle starts; refunds collateral in full.
function leaveBeforeStart() external;

/// Pay the current round's contribution. Requires prior ERC20 approve.
function contribute(uint8 round) external;

/// Place or improve a discount bid. Must strictly exceed the leading bid and be <= maxDiscountBps.
function placeBid(uint8 round, uint16 discountBps) external;
function withdrawBid(uint8 round) external;

/// Permissionless. Closes bidding, slashes non-payers, pays the winner, draws insurance if short.
function settleRound(uint8 round) external;

/// Winner pulls their pot (pull-payment; settleRound only credits).
function claimPayout(uint8 round) external;

/// After the final round settles and the member owes nothing.
function withdrawCollateral() external;

```

Events the frontend indexes / listens to for optimistic UI:

```

event MemberJoined(address indexed member, uint256 collateral, uint8 riskBand, bytes32 decisionRef);
event MemberLeft(address indexed member, uint256 refunded);
event ContributionPaid(uint8 indexed round, address indexed member, uint256 amount);
event BidPlaced(uint8 indexed round, address indexed bidder, uint16 discountBps);
event BidWithdrawn(uint8 indexed round, address indexed bidder);
event RoundSettled(uint8 indexed round, address indexed winner, uint16 discountBps,
    uint256 netPayout, uint256 redistributedPerMember, uint256 insuranceDrawn);
event CollateralDrawn(uint8 indexed round, address indexed member, uint256 amount);
event PayoutClaimed(uint8 indexed round, address indexed member, uint256 amount);
event CollateralReleased(address indexed member, uint256 amount);
event CircleCompleted(uint64 at);

```

Revert reasons → user-facing copy (map these in `src/lib/contracts/errors.ts`; never surface a raw revert string):

Custom error	Shown to user
<code>QuoteExpired()</code>	“Your deposit quote expired. Get a fresh one and try again.”
<code>BadAgentSignature()</code>	“We couldn’t verify the underwriter’s signature. Refresh and retry.”
<code>NotAMember()</code>	“You’re not a member of this circle.”
<code>AlreadyMember()</code>	“You’ve already joined this circle.”
<code>CircleFull()</code>	“This circle just filled up.”
<code>BiddingClosed()</code>	“Bidding closed for this round.”
<code>BidTooHigh(uint16 max)</code>	“The most you can bid here is {max/100}%.”
<code>BidNotImproving(uint16 leading)</code>	“You need to bid above {leading/100}% to take the lead.”
<code>AlreadyPaidOut()</code>	“You’ve already received your payout — you can’t bid again.”
<code>NothingDue()</code>	“You’re already paid up for this round.”

InsufficientAllowance()	“Approve USDC first, then retry.”
RoundNotSettled()	“This round hasn’t settled yet.”
CollateralLocked()	“Your deposit unlocks once the circle finishes.”

8.4 WhamReputation (soulbound)

```
function scoreOf(address who) external view returns (uint16); // 0–1000
function bandOf(address who) external view returns (uint8); // 0=A ... 3=D
function tokenIdOf(address who) external view returns (uint256); // 0 if none minted
function tokenURI(uint256 id) external view returns (string memory);
function statsOf(address who) external view returns (
    uint16 circlesCompleted, uint16 roundsOnTime, uint16 roundsLate,
    uint16 roundsMissed, uint16 defaults
);
event AttestationMinted(address indexed member, uint256 indexed tokenId);
event ScoreUpdated(address indexed member, uint16 oldScore, uint16 newScore, bytes32 reason);
```

Mock:

```
export const MOCK_REPUTATION_READS = {
    scoreOf: 410,
    bandOf: 1, // B
    tokenIdOf: 1042n,
    statsOf: { circlesCompleted: 2, roundsOnTime: 17, roundsLate: 1, roundsMissed: 0, defaults: 0 },
} as const;
```

No transfer functions exist. The profile page should say so plainly: *“This record is bound to your wallet. It can’t be sold or transferred.”*

8.5 WhamInsurancePool

```
function totalReserves() external view returns (uint256);
function committedCoverage() external view returns (uint256);
function availableCoverage() external view returns (uint256);
function coverageFor(address circle) external view returns (uint256);
function claimsPaid() external view returns (uint256 count, uint256 total);
event SkimReceived(address indexed circle, uint8 indexed round, uint256 amount);
event ClaimPaid(address indexed circle, uint8 indexed round, uint256 amount);
```

```
export const MOCK_INSURANCE_READS = {
    totalReserves: 48_620_000000n,
    committedCoverage: 31_400_000000n,
    availableCoverage: 17_220_000000n,
    coverageFor_circle: 1_600_000000n,
    claimsPaid: { count: 4n, total: 1_180_000000n },
} as const;
```

8.6 ERC-8004 registries

```
// IdentityRegistry
function resolveByAddress(address agent) external view returns (uint256 agentId, string memory domain);
function agentInfo(uint256 agentId) external view returns (address agentAddress, string memory domain, uint64 registeredAt);

// ReputationRegistry – decision attestations
function attestationCount(uint256 agentId) external view returns (uint256);
function attestationAt(uint256 agentId, uint256 i) external view returns (bytes32 ref, uint64 at, bytes32 outcome);
```

The frontend reads these on `/agent` to prove the identity is real on-chain rather than a claim in a marketing page. Show `agentId`, `domain`, `registeredAt`, and a link to the registry contract on Arbiscan.

```
export const MOCK_AGENT_READS = {
  resolveByAddress: { agentId: 1042n, domain: 'agent.wham.finance' },
  attestationCount: 412n,
} as const;
```

8.7 ERC-20 (USDC)

```
function balanceOf(address) external view returns (uint256);
function allowance(address owner, address spender) external view returns (uint256);
function approve(address spender, uint256 amount) external returns (bool);
function decimals() external view returns (uint8);
```

Approval policy: approve the exact amount needed for the action, not `type(uint256).max`. The audience is trust-sensitive and unlimited approvals are the thing security-literate judges will notice. Show the exact figure in the approval step copy: *“You’re approving exactly 500.00 USDC — the deposit plus your first contribution. Nothing more can be moved.”*

8.8 The two-step write pattern

Every value-moving action is approve-then-act. Build one hook and use it everywhere:

```
useApproveAndWrite({
  token, spender, amount,
  write: { address, abi, functionName, args },
})
// → { step: 'idle'|'approving'|'approved'|'writing'|'confirming'|'success'|'error',
//     needsApproval, approve(), execute(), reset(), txHash, error }
```

UI contract for the pattern:

1. Read `allowance` on mount. If sufficient, skip straight to step 2 and say so (“USDC already approved”).
2. Approve tx → wait 1 confirmation → **do not auto-fire the second tx**. Show “Approved. Now confirm the payment.” and let the user click. Auto-firing a second wallet prompt reads as malware to a cautious user.
3. Action tx → pending toast with Arbiscan link → 1 confirmation → invalidate the relevant TanStack Query keys → success state.
4. On failure at either step, keep the form state and offer retry.

Query keys to invalidate after each write:

Write	Invalidate
<code>join</code>	<code>['circle', id]</code> , <code>['circle', id, 'members']</code> , <code>['me', 'summary']</code> , <code>['me', 'circles']</code>
<code>contribute</code>	<code>['circle', id]</code> , <code>['round', id, n]</code> , <code>['me', 'summary']</code> , <code>['me', 'obligations']</code>
<code>placeBid</code> / <code>withdrawBid</code>	<code>['round', id, n]</code> , <code>['bids', id, n]</code> , <code>['me', 'summary']</code>
<code>claimPayout</code>	<code>['circle', id]</code> , <code>['me', 'summary']</code> , <code>['me', 'reputation']</code>
<code>withdrawCollateral</code>	<code>['circle', id]</code> , <code>['me', 'summary']</code> , <code>['me', 'reputation']</code>
<code>createCircle</code>	<code>['me', 'circles']</code> , <code>['circles']</code>

9. Page specifications

9.1 Landing — /

Auth: public · **Rendering:** static + ISR 60s · **Purpose:** convert a stranger into someone who understands what a savings circle is and trusts this one.

Content

Section	Copy
Hero eyebrow	Savings circles, on-chain
Hero headline	Save together. Get paid in turn.
Hero subhead	A group puts in a fixed amount each round. One member takes the whole pot. Everyone gets a turn. Wham makes it work with people you haven't met — with a deposit from every member, a transparent underwriter, and a shared safety net.
Hero CTA	Browse circles → /circles · Secondary: How it works → /how-it-works
Hero proof strip	<code>{circlesFormed}</code> circles · <code>{activeMembers}</code> members · <code>{totalDistributed}</code> paid out · <code>{membersWhoLostFunds}</code> members have lost money
Familiarity band	You may know this as a sandogh, tanda, iqub, chama, hui, or susu . Hundreds of millions of people already save this way. Wham is the same thing, with the trust part solved.

“How a circle works” — 4 steps

1. **Join a circle.** Pick a group size and a monthly amount. Post a refundable deposit sized to your track record.
2. **Bid for your turn.** Need the money early? Offer to take slightly less. Happy to wait? The discounts others give up get shared with you.
3. **Pay each round.** One fixed payment per round. Miss one and it comes out of your deposit — nobody else is left short.
4. **Get your deposit back.** Finish the circle clean and your deposit returns, along with a permanent record that lowers your deposit next time.

Six features — the exact framing to use

Feature	Headline	Body
Deposit	Everyone posts a deposit first	Before anyone touches the pot, every member locks 1-2× their contribution. It stays locked until they've paid into every remaining round. Miss a payment and it's taken from your deposit automatically — not chased, not negotiated.
Bidding	You choose when your turn comes	Payout order isn't random or first-come. Members bid how much of the pot they'll give up to be paid earlier. The highest discount wins the round — and the amount they gave up is shared among the members still waiting. Waiting pays.
Agent	An underwriter that shows its work	A registered on-chain agent sets each member's deposit individually, based on their history — circles completed, payments on time, defaults. Not one flat rule for everyone. Every decision is signed under the agent's own on-chain identity, so its accuracy is a matter of public record rather than a claim. <i>(Currently a rules-based v1. We say so on the agent's own page.)</i>
Reputation	Your record travels with you	Finish a circle without missing a payment and it's written to a permanent record bound to your wallet. That record is what unlocks larger circles and a smaller deposit next time. It can't be bought or transferred.
Insurance	A shared safety net	1.5% of each round's pot goes into a pool shared across circles. If someone defaults and their deposit doesn't cover it, the pool makes up the difference — so the members still waiting for their turn are the ones protected.

Graduated sizing

Start small, earn bigger

New wallets start in small, fully-collateralised circles. Bigger pots and lower deposits are earned, not granted. It keeps one unproven member from putting a large group at risk.

Trust section — “What happens if someone doesn’t pay” Prose + a small waterfall diagram: Their deposit → covers it → Deposit exhausted → insurance pool covers the rest → Members waiting are made whole → Defaulter's record is marked permanently. Copy: “There are three layers between a missed payment and you losing money. In {roundsSettled} rounds settled so far, we’ve reached layer two once and layer three never.”

Closing CTA: “Find a circle that fits.” → /circles

Reads: GET /v1/stats **Writes:** none **States:** if /v1/stats fails, render the page with the proof strip omitted entirely — never with zeroes or skeletons that persist.

9.2 How it works — /how-it-works

Auth: public · **Purpose:** the long-form version, for the sceptic and for judges.

Content sections, in order

1. **What a ROSCA is** — one paragraph, no crypto vocabulary. A worked example: “Eight friends, 50 USDC each, every two weeks. Round one, someone takes 400. Round two, someone else. After eight rounds, everyone has put in 400 and everyone has taken 400 out — but four of them got a lump sum months before they’d have saved it.”
2. **Why it breaks with strangers** — the honest problem statement. Informal circles run on social pressure. Scale past people who know each other and it collapses.
3. **What Wham replaces social pressure with** — deposit, bidding, agent, record, pool. Each linked to its own anchor.
4. **The bidding mechanism, worked through** — an interactive worked example (client island): sliders for pot size and discount, live output of “You’d receive X now instead of Y later. Each member still waiting receives Z extra.” Uses the same math as quotePayout.
5. **What the deposit costs you** — honest section. The deposit is capital you can’t use during the circle. Say it plainly, then explain why the alternative (no deposit) is worse.
6. **What we don’t do** — no interest, no lending, no leverage, no token. Explicit. This is the section that makes the rest credible.
7. **Fees** — 1.5% insurance skim per round, nothing else during the hackathon build. If a protocol fee is planned later, say so here rather than hiding it.

Reads: none (static) · **Writes:** none

9.3 Circle browser — /circles

Auth: public (viewer context when signed in) · **Purpose:** find a circle you can afford and are eligible for.

Content

- **Header:** “Open circles” + subhead “Every circle shows its size, its round amount, and the deposit you’d post. Nothing is hidden until you join.”
- **Filter rail** (left on desktop, sheet on mobile):
 - Contribution per round — range slider, presets 25 / 50 / 100 / 250 / 500 USDC
 - Group size — chips 4 / 6 / 8 / 10 / 12 / 20
 - Round length — chips Weekly / Fortnightly / Monthly
 - Tier — Starter / Standard / Trusted
 - Toggle: “Only circles I’m eligible for” (default ON when signed in)
 - Toggle: “Only circles I can afford” (checks wallet balance vs deposit + first contribution)
- **Sort:** Starting soon · Filling fast · Largest pot · Newest
- **Results grid** of CircleCard:
 - Name, tier chip, status chip
 - {contribution} / round · {memberTarget} members · {roundDuration}
 - **Pot: {grossPotPerRound}** — largest type on the card
 - “Your deposit: ~{estimate}” when signed in, “Deposit from {baseCollateral}” when not
 - Fill meter {memberCount}/{memberTarget} + “Starts {relative}”
 - CTA state machine driven entirely by viewer.cannotJoinReason :

- `null` → “Join circle”
- `reputation_too_low` → disabled “Needs a score of {min}” + link to `/app/reputation`
- `insufficient_balance` → “You’d need {amount}” (still clickable → detail page)
- `circle_full` → “Full — view circle”
- `already_member` → “You’re in this circle”

- **Empty state:** “No circles match those filters yet.” + button “Create a circle” + button “Clear filters”
- **Bottom band:** “Nothing fits? Start your own circle and invite people you know.” → `/app/circles/create`

Reads: `GET /v1/circles` (paginated, keep filters in URL search params so results are shareable) **Writes:** none **States:** loading = 6 skeleton cards; error = inline retry, never a blank page; `hasMore` = “Load more” button, not infinite scroll (judges scroll fast and infinite scroll hides the footer).

9.4 Circle detail (public view) — `/circles/{circleId}`

Auth: public · **Purpose:** everything a prospective member needs before committing money.

If `viewer.isMember` is true, redirect to `/app/circles/{circleId}`.

Layout: two columns desktop, stacked mobile. Left = content. Right = sticky action card.

Left column

1. **Header** — name, status chip, tier chip, `{memberCount}/{memberTarget} members`, creator avatar + link to `/members/{address}`, description.
2. **Terms table** — the contract, in plain rows:

Row	Value
Contribution	50.00 USDC every 14 days
Rounds	8
Pot per round	400.00 USDC
Your total in	400.00 USDC
Your total out	400.00 USDC minus any discount you bid
Deposit	1-2× contribution, set per member
Insurance skim	1.5% of each pot
Max bid discount	15%
Starts	15 Aug 2026

3. **Round timeline** — horizontal stepper of all rounds. Each node: index, dates, status, winner avatar + discount once settled. Settled nodes link to a round detail sheet showing pot breakdown and the settlement tx.
4. **Members table** — avatar, name/ENS (truncate address otherwise), risk band chip, reputation score, rounds paid, status chip, “paid out in round N” where applicable. Column tooltips explain band and score. Delinquent members shown honestly with a warning chip — hiding this destroys the trust proposition.
5. **Pot breakdown card** for the current round: gross pot → minus insurance skim → minus leading discount → net payout, with the redistribution per waiting member called out.
6. **Activity feed** — paginated, from `/activity`, each row linking to Arbiscan.
7. **Contracts card** — circle address, token address, agent address, all with copy buttons and Arbiscan links.

Right column — sticky action card

- **Not signed in:** “Connect wallet to see your deposit” → connect
- **Signed in, can join:** “Your deposit: {quotedCollateral}” + “First contribution: {contribution}” + “Total due now: {sum}” + risk band chip + “Why this amount?” link → `/agent/decisions/{id}` + primary CTA “**Join circle**” → `/app/circles/{id}/join`
- **Signed in, cannot join:** reason-specific card (see §9.3 CTA state machine), with a next-step link in every case
- **Circle active and viewer not a member:** “This circle has started. Browse open circles.”

Reads: `GET /v1/circles/{id}`, `/members`, `/rounds`, `/activity`; contract reads `roundInfo`, `leadingBid` for live bid state (poll 15s while `status === 'bidding'`) **Writes:** none from this page **States:** 404 → “That circle doesn’t exist or was cancelled.” + link to browse. Circle `cancelled` → banner explaining refunds were issued.

9.5 Agent transparency hub — /agent

Auth: public · **Purpose:** this is the page that wins or loses the ERC-8004 story with judges. Build it properly.

Content

- 1. **Header** — “The Wham underwriter” + subhead “One agent sets every deposit on Wham. It has its own on-chain identity, and every decision it makes is signed under that identity — so its judgement can be measured, not just trusted.”
- 2. **Identity card** — read live from the ERC-8004 IdentityRegistry, not from the API:
 - Agent ID 8004:421614:1042
 - Address, with Arbiscan link
 - Domain agent.wham.finance
 - Registered 14 Sep 2026
 - Registry contract, with Arbiscan link
 - Decisions issued / resolved
 - **Model badge:** “Rules-based v1.2.0” with the honesty tooltip from \$7.4
- 3. **Honesty panel** — a bordered callout, prominent, not buried: > **What this agent is, and isn’t.** Today it scores on explicit published rules — wallet age, circles completed, payment timing, concurrent commitments, circle size. It is not a trained model, and we’re not going to pretend otherwise three weeks into building it. What matters is the structure: every decision is signed, timestamped, and resolved against a real outcome. When a learned model replaces these rules, it inherits the same public scorecard.
- 4. **The rules, published** — a table of every factor, its trigger, and its bps effect. Same AgentFactor.key values the API returns.

Factor	Effect	Trigger
Completed circles	–1500 bps each, max –3000	Circle finished with zero missed rounds
Wallet history	–1000 bps	First on-chain activity older than 12 months
Clean payment streak	–500 bps	6+ consecutive on-time rounds
Circle size exposure	+2500 bps	Pot more than 2× the largest circle completed
Concurrent commitments	+1500 bps per active circle beyond the first	
Late payments	+2000 bps	Any late payment in the last 90 days
Prior default	+5000 bps, floor 20000	Any default on record
New wallet	+2500 bps	First on-chain activity under 3 months

- 5. **Scorecard** — AgentAccuracy rendered as a grouped bar chart: predicted vs realised default rate per band, plus calibration error and collateral coverage rate. Caption: “Predicted rates come from the band assigned at join time. Realised rates come from what actually happened. A well-calibrated underwriter has these two bars close together.”
- 6. **Recent decisions** — table of the last 25 public decisions: date, band, multiplier, circle, outcome chip, link to the decision page. Addresses shown truncated.
- 7. **Attestation record** — read attestationCount(agentId) from the ERC-8004 ReputationRegistry, link to the contract.

Reads: GET /v1/agent/identity , GET /v1/agent/stats , GET /v1/agent/decisions ; contract reads resolveByAddress , attestationCount
Writes: none

9.6 Decision detail — /agent/decisions/[decisionId]

Auth: public (shareable link) · **Purpose:** show exactly why one deposit was set the way it was.

Content

- 1. **Header** — “Deposit decision for {truncated address}” + date + outcome chip
- 2. **The number** — large: {collateralRequired} with the formula rendered as a visible calculation:

Contribution 200.00 USDC × multiplier 1.50× = 300.00 USDC deposit
- 3. **Multiplier waterfall** — the core visual. Horizontal bars from baselineMultiplierBps , each factor adding or subtracting, landing on finalMultiplierBps . Colour: increases in a muted amber, decreases in a muted green, neutral in grey. No red — this is not a warning.
- 4. **Factor list** — each AgentFactor as a card: label, detail sentence rendered verbatim, signed bps chip, evidence rows with

links.

5. **Provenance** — agent ID, model version, decision id, expiry, attestation tx hash → Arbiscan, attestation URI → raw JSON.
6. **Outcome** — once resolved: “This member completed the circle without missing a payment” / “This member missed 2 payments; their deposit covered the shortfall.” Then: “*The agent’s assessment is scored against this outcome on its public record.*” → `/agent`

Reads: `GET /v1/agent/decisions/{id}` **Writes:** none **States:** `outcome: 'pending'` → “Outcome pending — this circle is still running.” Expired quote → “This quote has expired. It’s kept here as a record.”

9.7 Public member profile — `/members/{address}`

Auth: public · **Purpose:** let members check each other, and let a member show their record externally.

Content

- Header: avatar, ENS/display name, truncated address with copy, “Member since {firstSeenAt}”
- **Score dial:** score / 1000, band chip, tier chip
- **Record grid:** circles completed · rounds on time · rounds late · rounds missed · defaults · total contributed
- **Soulbound note:** “*This record is bound to this wallet. It can’t be sold or transferred.*” + token id + mint tx link
- **Reputation timeline:** `ReputationEvent[]` as a vertical timeline with score deltas
- **Circles:** completed and active circles (public ones only), each linking to `/circles/{id}`
- **Agent decisions:** links to this address’s public decisions

Reads: `GET /v1/members/{address}`; contract reads `scoreOf`, `tokenIdOf`, `statsOf` **States:** unknown address → “No Wham record for this wallet yet.” with a “What’s a Wham record?” link, not a 404.

9.8 Insurance pool — `/insurance`

Auth: public · **Purpose:** prove the safety net has real money in it.

Content

- Header: “The shared safety net” + “*Every circle contributes 1.5% of each pot. If a member defaults and their deposit doesn’t cover it, this pool covers the rest — so the members still waiting for their turn don’t lose out.*”
- **Headline figures:** total reserves · available coverage · committed coverage · coverage ratio (with a plain-English gloss: “*For every 1 USDC of exposure across live circles, the pool holds {ratio} USDC*”)
- **Reserves over time** — area chart from `history`, with claims overlaid
- **Every claim ever paid** — full table (not “recent”): circle, round, shortfall, deposit drawn, pool paid, date, tx. Small numbers are fine; completeness is the point.
- **Waterfall explainer** — reuse the landing page diagram
- **Contract card** — pool address, Arbiscan link, live `totalReserves` read

Reads: `GET /v1/insurance/pool`; contract reads `totalReserves`, `availableCoverage`, `claimsPaid` **States:** zero claims → “*No claims have been paid yet. Every default so far has been covered by the defaulter’s own deposit.*” — that’s the good outcome, present it as such.

9.9 Join flow — `/app/circles/{circleId}/join`

Auth: session · **Purpose:** the highest-stakes screen in the product. Four steps, one per screen, no surprises.

A seat is reserved for 15 minutes on entry. Show the countdown from step 2 onward as a quiet line, not an alarm: “*Your seat and quote are held for 12:40.*”

Step 1 — Review the terms

Full terms table (same rows as \$9.4) plus a “**What you’re committing to**” block:

Over 8 rounds you’ll pay **400.00 USDC** in total, 50.00 USDC every 14 days. You’ll receive one pot of up to **394.00 USDC** — the exact amount depends on whether you bid a discount for an earlier turn. Your **300.00 USDC deposit** is locked until the circle finishes and comes back in full if you don’t miss a payment.

Checkbox, required: “*I understand missed payments are taken from my deposit automatically.*” CTA: **Continue**

Step 2 — Your deposit

The agent quote, explained before it's charged.

- Large figure: `{collateralRequired}`
- The visible calculation: `50.00 USDC × 2.00× = 100.00 USDC`
- Risk band chip + one-line meaning: *"Band B — established wallet, short circle history."*
- **Factor list** — the same `AgentFactor` cards as \$9.6, collapsed to the top 3 by default with "See all {n} factors"
- Link: "How the underwriter works" → `/agent`
- Footer line: *"This deposit isn't a fee. You get all of it back when the circle finishes, provided you've paid every round."*

CTA: **Continue** · Secondary: **Get a fresh quote** (re-runs `POST /v1/agent/quote`; visible only when the quote is within 3 minutes of expiry)

Step 3 — Approve USDC

- Amount breakdown table: Deposit `300.00` + First contribution `200.00` = **Total 500.00 USDC**
 - Balance line: "Your balance: 812.40 USDC" — if insufficient, block with "You need {shortfall} more USDC" + faucet link on testnet
 - Copy: *"You're approving exactly 500.00 USDC — the deposit plus your first contribution. Nothing more can be moved from your wallet by this circle."*
 - If `allowance >= total`: skip with "USDC already approved ✓" and auto-advance
 - CTA: **Approve 500.00 USDC** → wallet prompt → confirming state → "Approved ✓"
-

Step 4 — Join

- Summary of everything above, compact
- CTA: **Join circle** → `join(quote.message, quote.signature)`
- Pending: "Confirming on Arbitrum..." + tx hash link + *"Don't close this tab — we'll confirm your seat as soon as it lands."*
- On confirmation: `POST /v1/circles/{id}/confirm-join` then redirect to `/app/circles/[id]` with a success toast and a one-time celebratory state on the member view

Reads: `POST /v1/circles/{id}/join-intent` (on mount), `GET /v1/circles/{id}`; contract `allowance`, `balanceOf`, `agentNonce`, `memberOf`

Writes: `USDC.approve(circle, total)`, `WhamCircle.join(quote, sig)`, `POST /v1/circles/{id}/confirm-join` **States / edge cases:** - Reservation expires mid-flow → modal: "Your seat was released. Circles fill quickly — check if there's still room." + re-run `join-intent` - `QUOTE_EXPIRED` on the join tx → step 2 with a fresh quote, form state preserved - Circle fills while the user is in the flow → step 1 blocked with `CIRCLE_FULL` copy - User abandons → `DELETE /v1/circles/{id}/join-intent` on unmount (and `navigator.sendBeacon` on unload) - Wrong chain → blocking banner, all CTAs disabled

9.10 Dashboard — `/app`

Auth: session · **Purpose:** answer "what do I owe, and when?" in under two seconds.

Content

1. **Greeting row** — "Good morning, {displayName ?? truncated address}" + wallet balance chip
2. **Next action card** — the single most prominent element on the page. Driven by `me.summary.nextObligation`:
 - Contribution due: **"Pay 50.00 USDC to Diaspora Circle #3 — due in 12 days"** + inline **Pay now** button (opens the contribute flow in a sheet, no page change)
 - Overdue: same card in warning treatment + **"{amount} will be taken from your deposit on {date}"** if this isn't paid."
 - Bid window open: **"Bidding closes in 2 days** for Diaspora Circle #3 round 4" + "Place a bid"
 - Payout ready: **"394.00 USDC is waiting for you"** + "Claim payout"
 - Nothing due: "You're all paid up. Next payment {date}."
3. **Position strip** — three figures: Committed `{totalCommitted}` · Deposit locked `{collateralLocked}` · Received `{receivedToDate}`. Each with a one-line gloss.
4. **My circles** — horizontal cards, one per active circle: name, current round, your status chip, next due date, mini progress bar of rounds settled, "paid ✓ / due" indicator for the current round
5. **Open bid windows** — compact list from `openBidWindows`: circle, round, closes in, your bid, leading bid, "You're leading" /

“Outbid” chip + “Adjust bid”

6. **Reputation strip** — score, band, 30-day delta, progress bar to next tier, “See your record” → `/app/reputation`
7. **Recommended circles** — up to 4 cards from `/v1/circles/recommended`, only shown when the user has fewer than 3 active circles

Reads: `GET /v1/me/summary`, `GET /v1/me/circles?status=active`, `GET /v1/circles/recommended`; contract `balanceOf` **Writes:** contribute / bid / claim via sheets (same handlers as §9.13) **States:** brand-new user with no circles → replace the whole grid with an onboarding panel: “You haven’t joined a circle yet” + two CTAs (Browse circles / Create a circle) + a compact “how it works” recap.

9.11 Onboarding — `/app/onboarding`

Auth: session, `firstLogin: true` only · **Purpose:** three quick steps, skippable.

1. **Name yourself** — display name input, optional. “Other members in your circles will see this. Leave it blank to show your wallet address.”
2. **Stay on top of payments** — email and/or Telegram, reminder timing chips (3 days / 1 day / 4 hours before). Copy: “Missing a payment costs you money out of your deposit. We’ll remind you.” Both optional; in-app notifications are always on.
3. **Find your first circle** — 3 recommended starter-tier circles + “Browse all”

Skip link on every step. On completion `PATCH /v1/me` with `firstLogin: false`.

Reads: `GET /v1/circles/recommended` · **Writes:** `PATCH /v1/me`

9.12 My circles — `/app/circles`

Auth: session

Content: tabs — **Active ({n})** · **Forming ({n})** · **Completed ({n})**

Each row (not card — this is a working list): - Name, tier chip, current round `4 of 8` - Your status chip - This round: paid ✓ / due {amount} by {date} / bidding open - Deposit: {posted}, {slashed} drawn if any - Progress bar - Row CTA driven by `viewer.nextActionLabel`

Completed tab additionally shows: total in, total out, discount bid, deposit returned date, “View record” → circle page.

Reads: `GET /v1/me/circles?status=` **Empty:** per-tab — Active: “No active circles.” Forming: “Nothing forming.” Completed: “You haven’t finished a circle yet. Your first completed circle is what unlocks lower deposits.”

9.13 Member circle view — `/app/circles/{circleId}`

Auth: session + membership (redirect non-members to the public page) · **Purpose:** everything §9.4 shows, plus the ability to act.

Layout: same two-column as §9.4. The right column becomes an **action stack** instead of a join card.

Action stack — ordered by urgency, only the applicable ones render:

1. **Pay this round** (`viewer.canContribute && !hasContributed`)
 - “Round 4 · 50.00 USDC due 9 Oct”
 - Warning when within 48h: “If this isn’t paid by {date}, 50.00 USDC will be taken from your deposit.”
 - CTA **Pay 50.00 USDC** → approve-then- `contribute(round)` sheet
2. **Bidding is open** (`viewer.canBid`)
 - Closes in {countdown}
 - Current leading bid: 8.5% (Sepehr) — or “No bids yet”
 - **Bid slider** 0 → `maxDiscountBps`, stepped at 0.25%
 - Live output as the slider moves, using `quotePayout(round, bps)`:
 - “You’d receive **360.00 USDC** instead of 394.00”
 - “You’d give up **34.00 USDC**”
 - “Each of the 4 members still waiting receives **8.50 USDC** extra”
 - Validation: must exceed leading bid, must be ≤ max. Disabled with `BidNotImproving` copy otherwise.
 - CTA **Place bid / Raise bid** · Secondary **Withdraw bid**
 - Once outbid, a notification fires and this card shows “You’ve been outbid at 9.0%”

3. Claim your payout (`viewer.canClaimPayout`)

- “You won round 4. **360.00 USDC** is waiting.”
- CTA **Claim 360.00 USDC** → `claimPayout(round)`
- Post-claim reminder, prominent: “*You still owe 50.00 USDC every round until the circle finishes. Your deposit stays locked until then.*” — this is the single most important piece of copy in the app for default prevention.

4. Withdraw your deposit (`viewer.canWithdrawCollateral`)

- “The circle is complete. Your deposit of 100.00 USDC is unlocked.”
- CTA **Withdraw 100.00 USDC** → `withdrawCollateral()`

5. Your standing (always)

- Deposit posted / drawn / remaining
- Rounds paid, rounds missed
- Your payout round if determined
- Link: “Why is my deposit this amount?” → `/agent/decisions/[id]`

6. Delinquency banner (when `memberStatus === 'delinquent'`) — top of page, warning treatment:

- “You missed round 3. 50.00 USDC was taken from your deposit. **50.00 USDC of deposit remains** — if it runs out you’ll be marked as defaulted and your record will show it permanently.”
- CTA **Pay round 4 now** + link “Top up my deposit” if the contract supports it

Left column additions vs §9.4: - **Your rounds** table: every round, your payment status, timestamp, tx hash - **Your bids** table: round, discount bid, outcome - Members table gains a “you” marker

Reads: `GET /v1/circles/{id}, /members, /rounds, /rounds/{n}/bids, /activity`; contract `memberOf, roundInfo, leadingBid, quotePayout, amountDue, hasContributed, withdrawableCollateral, allowance` **Writes:** `approve + contribute(round), placeBid(round, bps), withdrawBid(round), claimPayout(round), withdrawCollateral()` **Polling:** while `round.status === 'bidding'`, poll `leadingBid` and `/bids` every 15s. Everything else refetches on window focus and after tx confirmation. **States:** circle `unwinding` → banner explaining what’s happening and what the member’s exposure is; circle `cancelled` → refund status card.

9.14 Create a circle — `/app/circles/create`

Auth: session · **Purpose:** let a user start a circle for a group they know. Four steps.

Step 1 — Shape - Contribution per round (amount input, USDC) - Number of members (4–20, gated by tier: starter caps at 8, standard at 12, trusted at 20 — show the cap and why) - Round length: Weekly / Fortnightly / Monthly - **Live summary panel, always visible:** “8 members × 50.00 USDC every 14 days. Pot of 400.00 USDC per round. The circle runs 8 rounds and finishes around 5 Dec 2026. Total commitment per member: 400.00 USDC.”

Step 2 — Rules - Bidding enabled (default on) + max discount (default 15%, slider 5–25%) - Insurance skim — displayed, not editable: 1.5% - Base deposit multiplier — slider 1.0×–2.0×, default 2.0× for starter. Copy: “*This is the starting point. The underwriter adjusts it up or down per member based on their record.*” - Visibility: Public (listed in the browser) / Invite only (link + code) - Start date

Step 3 — Review & deploy - Full terms table - **Your own deposit** — the creator joins on the same terms, so run `POST /v1/agent/quote` here and show it. Copy: “*You post a deposit like everyone else. The underwriter doesn’t give the organiser a discount.*” - Predicted contract address from `predictAddress` - CTA **Deploy circle** → `createCircle(deployParams, salt)` → on receipt, `POST /v1/circles/{id}/link-deployment`

Step 4 — Invite - Invite link + code, copy buttons, share sheet on mobile - “You’ll need {`memberTarget - 1`} more members before the circle can start.” - CTA **Join my own circle** → straight into §9.9 step 2

Reads: `POST /v1/circles, POST /v1/agent/quote, GET /v1/me/reputation` (for tier caps); contract `predictAddress` **Writes:** `createCircle, POST /v1/circles/{id}/link-deployment` **States:** tier-locked options shown disabled with the requirement, not hidden — it’s a growth loop.

9.15 Payment calendar — `/app/calendar`

Auth: session · **Purpose:** see every obligation across every circle in one place.

Content

- **View toggle:** Month grid / List (default List on mobile, Month on desktop)
- **Month grid:** each day cell shows obligation pills, colour-coded by type. Contribution = solid, bid window = outlined, payout = accent, collateral release = muted. Day click → side sheet with that day’s obligations and inline actions.
- **List view:** grouped by week, each row: date, circle name, type, amount, status chip, action button

- **Warning treatment:** any `autoDeductRisk: true` obligation within 72h renders with a warning border and the line “Auto-deducted from your deposit if unpaid”
- **Summary strip:** “This month: 3 payments totalling 300.00 USDC · 1 payout expected”
- **Export:** “Add to calendar” → generate an `.ics` client-side from `Obligation[]` with 3-day and 1-day alarms. Cheap to build, disproportionately useful for the target user.

Reads: `GET /v1/me/obligations?from=&to=` **Writes:** contribute / bid / claim via the same sheets as §9.13 **States:** empty month → “Nothing due this month.” Timezone rendered per `UserProfile.timezone` with a footer note: “Times shown in {timezone}.” Deadlines are enforced on-chain in UTC.”

9.16 My reputation — `/app/reputation`

Auth: session · **Purpose:** make the incentive legible — show exactly what good behaviour buys.

Content

1. **Score dial** — score /1000, band chip, 30-day delta
2. **What this gets you right now:**
 - “Your deposit multiplier: **1.50×**” with a comparison line: “A new wallet posts 2.00×. A trusted member posts 0.75×.”
 - “Circles you can join: up to **12 members**”
 - “Circles you can create: **starter and standard**”
3. **Tier ladder** — three tiers, current one marked, each showing requirements and unlocks. Next tier gets a progress bar: “1 more completed circle and 6 more clean rounds.”
4. **Record grid** — same figures as §9.7
5. **Reputation timeline** — `ReputationEvent[]`, each with score delta and the circle that caused it
6. **Score chart** — line chart of `scoreAfter` over time
7. **Attestation card** — token id, mint tx, “bound to this wallet, can’t be sold or transferred”, link to the public profile with a copy-link button (people will want to share this)
8. **How the score moves** — a plain table: on-time round +8 · late round −15 · missed round −60 · circle completed clean +120 · default −400. Publishing this is the point.

Reads: `GET /v1/me/reputation`; contract `scoreOf`, `bandOf`, `tokenIdOf`, `statsOf` **States:** no history → “Your record starts with your first circle.” + the tier ladder shown as a preview.

9.17 Activity — `/app/activity`

Auth: session · **Purpose:** a personal ledger and notification history in one.

Content

- Tabs: **All** · **Transactions** · **Notifications**
- Filter chips: circle, type, date range
- Transaction rows: icon, summary, amount (signed — in/out), circle, timestamp, tx hash → Arbiscan
- Notification rows: severity icon, title, body, timestamp, link, read/unread
- “Mark all as read”
- **Export CSV** — the target user may need records for their own bookkeeping. Columns: date, circle, type, amount, direction, tx hash.

Reads: `GET /v1/me/notifications`, `GET /v1/circles/{id}/activity` merged client-side per circle membership **Writes:** `POST /v1/me/notifications/read`

9.18 Settings — `/app/settings`

Auth: session

Sections

1. **Profile** — display name, avatar preview (deterministic from address), ENS shown read-only if resolved
2. **Notifications** — channel toggles (in-app always on, email, Telegram), reminder timing multi-select `[72, 24, 4]` hours, per-type toggles (payment due, bid window, outbid, round settled, payout ready, collateral drawn)
3. **Email** — input + “Send verification” → verified badge
4. **Telegram** — “Connect Telegram” → deep link to the bot with a one-time code
5. **Regional** — timezone (IANA select, default from `Intl.DateTimeFormat().resolvedOptions().timezone`), language (English / فارسی —

- ship the toggle even if `fa` is partial; the target user segment notices)
6. **Wallet** — connected address, “Sign out”, explainer: *“Signing out ends your session. It doesn’t affect your circles, your deposit, or your record — those live on-chain.”*
 7. **Danger zone** — nothing destructive is possible (no account deletion while circles are active). Say so: *“You can’t delete your record while you have active circles. Your on-chain record is permanent by design.”*

Reads: `GET /v1/me` · **Writes:** `PATCH /v1/me`

9.19 Faucet — `/app/faucet` (testnet only)

Auth: session · Hidden entirely when `chainId === 42161`.

- “This is the Arbitrum Sepolia demo. Grab test USDC to try Wham.”
- Balance display + CTA **Get 5,000 test USDC** → mock USDC `mint(msg.sender, 5000e6)`
- Link to an Arbitrum Sepolia ETH faucet for gas
- Rate limit: once per 24h per address, enforced by the mock token

Put a link to this in the app sidebar footer on testnet. Judges will not go hunting.

10. Cross-cutting patterns

10.1 Transaction lifecycle UX

One component, `<TxButton>`, wraps every write. States:

State	Button	Feedback
<code>idle</code>	Action label	—
<code>simulating</code>	“Checking...” disabled	—
<code>awaiting_signature</code>	“Confirm in your wallet” disabled	Sheet stays open
<code>pending</code>	“Sending...” disabled	Toast with tx hash + Arbiscan link
<code>confirming</code>	“Confirming...” disabled	Toast updates: “1 of 1 confirmations”
<code>success</code>	“Done ✓” then reset	Success toast + query invalidation
<code>error</code>	Original label re-enabled	Inline error using the §8.3 mapping, form state preserved

Always `simulateContract` before `writeContract`. A simulated revert caught before the wallet prompt is a far better experience than a failed transaction, and it’s where the §8.3 error mapping actually fires.

Never auto-chain two wallet prompts (see §8.8).

10.2 Money formatting

```
formatAmount('50000000', 6) // "50.00"
formatAmount('50000000', 6, { symbol: true }) // "50.00 USDC"
formatCompact('184300000000', 6) // "184.3K USDC" — headline stats only
```

Rules: - Always 2 decimal places for USDC. Never more, never fewer. - Thousands separators per locale. - Never round in a way that understates an obligation — round **up** on amounts owed, **down** on amounts receivable. A user who sends the displayed amount must never be one cent short. - Compact notation only in marketing stats. Never on an amount someone has to pay. - Basis points render as percentages with one decimal: `850` → `8.5%`.

10.3 Date and time

- Store and compare UTC. Render in `UserProfile.timezone`.
- Deadlines within 7 days → relative (“in 3 days”, “in 12 hours”). Beyond that → absolute (“9 Oct 2026”).
- Deadlines within 24 hours → live countdown, `mm:ss` precision under one hour.
- Every relative time has the absolute time in a `title` attribute.
- Footer note anywhere deadlines appear: *“Deadlines are enforced on-chain in UTC.”*

10.4 Address display

- Truncate as 0x7A3f...5A40 (4 + 4).
- Prefer ENS, then displayName, then truncated address. Never show a bare address when a name exists.
- Every address is copyable and links to Arbiscan.
- Deterministic avatars seeded from the address so members are visually distinguishable in tables.

10.5 Loading, empty and error states

Every data surface needs all four defined before it ships:

State	Rule
Loading	Skeletons matching final layout dimensions. No spinners on page-level loads.
Empty	A sentence explaining why it's empty + one action. Never just "No data".
Error	The ApiError.message verbatim + a retry button. Never a raw stack or status code.
Partial	If contract reads fail but API data succeeds (or vice versa), render what you have and mark the rest "Couldn't load — retry". Do not blank the page.

10.6 Trust-preserving copy rules

The audience is managing household savings. These are non-negotiable:

- **No hype vocabulary.** No "yield", "APY", "farming", "degen", "moon", "alpha". Wham pays no interest and must never imply it does.
- **Say what a thing costs before you ask for it.** Every CTA that moves money states the amount in the button label.
- **Name the downside.** The deposit is locked capital. Missed payments are auto-deducted. Say both plainly and repeatedly.
- **Never hide a delinquent member** from a circle's public view.
- **Never imply the agent is smarter than it is.** modelKind: 'rules' renders as "Rules-based v1" wherever the agent is named.
- **"Wham" plays only at the payout moment.** The success state after claimPayout can be delightful. Nothing else in the product is jokey.

10.7 Persian / RTL readiness

Ship the fa locale toggle even if translation is partial. Practical requirements: - dir="rtl" on <html> when locale === 'fa'; Tailwind logical properties (ms-, me-, ps-, pe-) throughout — no ml- / mr- . - Latin numerals for money even in Persian (financial convention), Persian numerals acceptable elsewhere. - Vazirmatn for fa, your Latin face for en . - Round timelines and waterfall charts must mirror. Test the bidding slider in RTL specifically — sliders are where RTL breaks.

10.8 Accessibility floor

- All interactive elements keyboard reachable; the bid slider needs arrow-key stepping.
- Colour is never the only signal — status chips carry text, the waterfall carries signed numbers.
- WCAG AA contrast on the banking palette, including the amber warning treatments on light backgrounds.
- Live regions announce tx state changes.
- Countdown timers get an aria-live="polite" region updated at most once per minute.

11. Build order

Three weeks, one frontend developer, with the .NET service and contracts landing in parallel. Build against mocks from day one.

Week 1 — Skeleton and trust surfaces

#	Task
1	Next.js scaffold, Tailwind tokens from the homepage design, layout shells
2	src/types + Zod schemas (§6) — all of them, before any UI

3	<code>src/mocks</code> + MSW handlers from every fixture in §7 and §8
4	wagmi + RainbowKit + chain guard banner
5	SIWE end-to-end including the account-switch handler (§3.3)
6	<code>/</code> landing with real copy from §9.1
7	<code>/circles</code> browser + <code>/circles/{id}</code> public detail

Week 2 — The money paths

#	Task
8	<code>useApproveAndWrite</code> + <code><TxButton></code> + error mapping (§8.3, §10.1)
9	Join flow, all four steps, including expiry and failure paths (§9.9)
10	<code>/app</code> dashboard + member circle view (§9.13)
11	Contribute, bid, claim, withdraw — all four writes wired
12	<code>/app/calendar</code> + <code>.ics</code> export
13	Swap MSW for the real .NET service, endpoint by endpoint

Week 3 — The differentiator and the polish

#	Task
14	<code>/agent</code> hub + <code>/agent/decisions/{id}</code> waterfall — this is the submission's centrepiece, budget real time for it
15	<code>/app/reputation</code> + <code>/members/{address}</code>
16	<code>/insurance</code>
17	Create-circle flow
18	Settings, activity, onboarding, faucet
19	Empty/error/loading pass across every surface (§10.5)
20	Seed a demo circle mid-rotation on Sepolia with 8 funded wallets, one delinquent, one settled round with a real discount — the demo needs a circle that has already lived a little
21	Record the walkthrough

11.1 Demo-day checklist

- ☐ A circle in `bidding` state with two competing bids, so the slider does something live
- ☐ A settled round showing discount redistribution to waiting members
- ☐ One delinquent member with a visible collateral draw
- ☐ Two agent decisions with visibly different multipliers for different wallets — the whole pitch in one screenshot
- ☐ A resolved decision with an outcome, linked from `/agent`
- ☐ Insurance pool with at least one real claim
- ☐ Faucet working, linked in the sidebar
- ☐ Every Arbiscan link resolving on Sepolia
- ☐ `membersWhoLostFunds: 0` on the landing page, and true

11.2 Known gaps to decide before building

- Collateral top-up** — §9.13 references a “Top up my deposit” action for delinquent members. Confirm whether `WhamCircle` exposes it; if not, cut the CTA rather than shipping a dead button.
- Who calls `settleRound`** — it's permissionless, but something must call it on time. Either a backend keeper or a visible “Settle round” button for any member once `contributionDueAt` passes. Decide, because the UI differs.
- Bid ties** — if two members bid the same discount, the contract needs a deterministic tiebreak (earliest bid). Confirm and reflect it in the bid ladder copy.
- Payout-then-default** — the member who takes an early pot at a high discount is the highest-risk member in the circle. Confirm the contract locks their full remaining obligation against their deposit at payout time, and if it does, say so on the bidding screen.
- Invite-only join** — `POST /v1/circles/{id}/join-intent` needs to accept an invite code for private circles. Not currently in the endpoint signature.

